



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Татјана Гавриловић

**Пројектовање и имплементација
хибридног текстуалног претраживача
над транскриптима YouTube подкаста
на српском језику**

ЗАВРШНИ РАД

Мастер академске студије

Нови Сад, 2026



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - мастер рад
Аутор, АУ:	Татјана Гавриловић
Ментор, МН:	Др Бранко Милосављевић, редовни професор
Наслов рада, НР:	Пројектовање и имплементација хибридног текстуалног претраживача над транскрипцијама YouTube подкаста на српском језику
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	10/79/51/2/38/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	У овом раду је реализован систем за текстуалну претрагу транскриптата <i>YouTube</i> подкаста на српском језику, који омогућава проналажење релевантних епизода и временских исјечака на основу корисничког упита. Корпус обухвата 15.889 транскрибованих епизода са 32 <i>YouTube</i> канала. Систем комбинује лексичку претрагу засновану на алгоритму <i>BM25</i> и семантичку претрагу засновану на моделу <i>multilingual-e5-base</i> , док су њихови резултати обједињени методом <i>Reciprocal Rank Fusion (RRF)</i> . Евалуација на 20 ручно означених упита је показала да семантичка претрага боље проналази релевантан садржај када не постоји непосредно лексичко подударање, док хибридни приступ побољшава његово позиционирање у ранг-листи.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	29.09.2026
Чланови комисије, КО:	Председник: Др Горан Слађић, редовни професор
	Члан: Др Данијела Боберић Крстићев, редовни професор
	Члан:
Члан, ментор:	Др Бранко Милосављевић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Tatjana Gavrilović
Mentor, MN :	Branko Milosavljević, Phd., full professor
Title, TI :	Design and Implementation of a Hybrid Text Search Engine for Serbian-Language YouTube Podcast Transcripts
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	10/79/51/2/38/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Template, thesis, tutorial
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This thesis presents a system for text search over Serbian-language YouTube podcast transcripts, enabling users to find relevant episodes and corresponding time segments based on a textual query. The corpus consists of 15,889 transcribed episodes from 32 YouTube channels. The system combines lexical search based on the BM25 algorithm with semantic search based on the multilingual-e5-base model, while their results are combined using Reciprocal Rank Fusion (RRF). An evaluation conducted on 20 manually annotated queries showed that semantic search is more effective at retrieving relevant content when there is no direct lexical match, while the hybrid approach improves the ranking of relevant results.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	29.09.2026
Defended Board, DB :	President: Goran Sladić, Phd., full professor
	Member: Danijela Boberić Krstićev, Phd., full professor
	Member:
Member, Mentor:	Branko Milosavljević, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
2	Изазови обраде српског језика као природног језика	3
2.1	Историјски разлози за ограничене језичке ресурсе	3
2.2	Писмо, дијалекти и фонолошке варијанте	3
2.3	Морфолошка сложеност српског језика	4
2.4	Вишезначност и семантичка сложеност	5
3	Преглед приступа	7
3.1	Припрема података и индексирање	7
3.2	Обрада корисничког упита	8
4	Архитектура система	9
4.1	База података	9
4.2	Претраживачки систем	9
4.3	Позадински сервис	10
4.4	Кориснички интерфејс	10
4.5	Мрежа и покретање система	11
5	Транскрипција и припрема корпуса	13
5.1	Обим корпуса	13
5.2	Прикупљање и транскрипција	13
5.3	Формат складиштења	14
5.4	Складиштење у <i>PostgreSQL</i> бази	15
5.5	Модел података	15
5.6	Функција за увоз података у базу	16
6	Лексичка претрага (<i>Elasticsearch</i>)	19
6.1	Шта је <i>Elasticsearch</i>	19
6.2	<i>Elasticsearch</i> наспрам релационе базе података	19
6.3	Дистрибуирана архитектура	19
6.4	Како <i>Elasticsearch</i> чува и претражује податке?	20
6.5	<i>BM25</i> и рангирање резултата	20
6.6	Ниво индексирања	21
6.7	Мапирање	21
6.8	Изазови српског језика у <i>Elasticsearch</i> -у	23
6.9	Анализатор	23
6.10	Алгоритам индексирања	26
6.11	Алгоритам претраге	28
7	Семантичка (векторска) претрага	33
7.1	Векторске репрезентације текста	33
7.2	Мјера сличности: <i>dot product</i> , <i>cosine similarity</i> и еуклидско растојање	34
7.3	Нормализација вектора и избор мјере сличности	35
7.4	Проблем приближне претраге и <i>HNSW</i>	35
7.5	<i>pgvector</i> као подршка за векторску претрагу	35
7.6	Избор <i>embedding</i> модела	36
7.7	Текстуални префикси	36
7.8	Грануларност и <i>chunking</i>	37
7.9	Припрема базе: <i>HNSW</i> индекс	38

7.10	Алгоритам индексирања (<i>offline pipeline</i>)	38
7.11	Алгоритам претраге (<i>online pipeline</i>)	41
7.12	Ограничења векторске претраге	44
7.13	Могућа унапређења	45
8	Хибридизација резултата претраге (<i>RRF</i>)	47
8.1	<i>Reciprocal Rank Fusion</i>	47
8.2	Избор <i>RRF</i> -а у односу на алтернативне приступе	48
8.3	Архитектура: три ендпоинта	49
8.4	Ниво фузије резултата	49
8.5	Алгоритам фузије	49
8.6	Пагинација хибридних резултата	51
8.7	Кеширање	52
8.8	Филтрирање након фузије	53
8.9	Претрага без текстуалног упита	53
8.10	Приказ релевантних исјечака по епизоди	54
8.11	Ограничења и правци даљег развоја	55
9	Евалуација система	57
9.1	Евалуациони скуп	57
9.2	Мјере евалуације	57
9.3	Резултати	58
9.4	Анализа појединачних упита	59
9.5	Ограничења евалуације	60
10	Закључак	61
	Списак слика	63
	Списак листинга	65
	Списак табела	67
	Списак коришћених скраћеница	69
	Списак коришћених појмова	71
	Биографија	75
	Литература	77

Енорман раст и популаризација подкаста (енг. *podcasts*) и *YouTube* садржаја су резултат комбинације технолошке демократизације, промјене навика корисника, те развоја нових начина дистрибуције и зараде. Међутим, док количина доступног садржаја континуирано расте, могућност прецизног проналажења релевантних тренутака унутар дугих епизода заостаје за овим развојем. Корисник који тражи конкретну информацију у епизоди тренутно нема начин да избјегне преслушавање читавог садржаја, нити да добије хронолошки организован преглед тема унутар епизоде са могућношћу директног преласка на релевантан исјечак.

Слични приступи претрази подкаст садржаја су развијани за енглески језик. Платформа *Spotify* је развила семантичку претрагу подкаста засновану на моделу обраде природног језика (ОПЈ, енг. *Natural Language Processing, NLP*) и приближној претрази (к) најближих сусједа (енг. *Approximate Nearest Neighbor, ANN*) [1]. Ипак, ријеч је искључиво о семантичкој претрази, а не о хибридном приступу који комбинује семантичку и лексичку претрагу. Пројекат *TREC Podcasts Track* је представљао истраживачки оквир за претрагу подкаст сегмената, покренут 2020. године и настављен 2021. године [2], али је обустављен, а одговарајући скуп података је укинут 2023. године [3].

Иако постоје примјери примјене семантичке претраге на српском језику у другим доменима (нпр. правна пракса [4]) и активан истраживачки рад на текстуалним *NLP* ресурсима за српски (нпр. *COMtext.SR* [5]), ни у једном пронађеном извору није идентификовано рјешење за семантичку или хибридну претрагу говорног, односно подкаст садржаја на српском језику са мапирањем на конкретан тренутак у оквиру епизоде.

Посебан изазов представља и сам српски језик, који припада језицима са ограниченим ресурсима у области обраде природног језика. Доступност квалитетних и обимних корпуса за српски знатно је мања у односу на доминантне језике. Због тога вишејезични модели, кроз пренос знања из других језика, не могу у потпуности обухватити језичке специфичности српског језика, укључујући његову морфосинтаксичку сложеност и различите језичке варијанте [6].

Циљ овог рада је дизајн инфраструктуре и имплементација система за хибридну претрагу *YouTube* садржаја на српском језику, који комбинује лексичку и семантичку претрагу путем методе реципрочне фузије рангова (енг. *Reciprocal Rank Fusion, RRF*), уз очување грануларности на нивоу једног или више сегмената ради навигације до тачног или приближног тренутка у епизоди.

Сљедеће поглавље разматра специфичне изазове обраде српског језика као природног језика, са освртом на историјске околности развоја језичких ресурса, те његово морфолошко богатство које утиче на изградњу система за претрагу. Треће поглавље даје преглед цјелокупног приступа и представља основне токове система, од припреме и индексирања података до обраде корисничког упита и добијања коначних резултата, чиме се пружа оквир за детаљнији опис у наредним поглављима. У глави четири описани су архитектура система, коришћене технологије и организација контејнеризованог окружења. Глава пет посвећена је транскрипцији и припреми корпуса, укључујући поступак добијања транскрипата и дизајн структуре података чуване у релационој бази. У шестом поглављу описује се лексичка претрага заснована на *Elasticsearch* систему, док се поглавље седам бави семантичком, односно векторском претрагом. Глава осам представља поступак хибридизације резултата лексичке и семантичке претраге примјеном методе *Reciprocal Rank Fusion (RRF)*, док је глава девет посвећена евалуацији система. Закључно, десето поглавље заокружује предмет овог рада, сумира постигнуте резултате и указује на могуће правце даљег развоја.

Глава 2

Изазови обраде српског језика као природног језика

Обрада природних језика (енг. *Natural Language Processing, NLP*) се бави рачунарском обрадом и разумевањем текста написаног на природним језицима, тј. језицима које људи користе у међусобној комуникацији (нпр. српски језик, енглески језик, итд.). За разлику од програмских језика, код којих су начини изражавања унапријед строго дефинисани, природни језици немају таквих инхерентних ограничења [7].

Развој *NLP* технологија дуго је био усмјерен првенствено на енглески језик, који и данас има доминантну улогу у *NLP* истраживањима [8]. Ова ситуација је дјелимично посљедица доминантне улоге енглеског језика у међународној комуникацији, што је довело и до веће комерцијалне заинтересованости за развој *NLP* рјешења за енглеско говорно подручје [7]. Посљедице су посебно изражене код језика попут српског, за које не постоје ресурси по обиму и квалитету упоредиви са онима доступним за енглески језик. Недостатак великих и језички анотираних корпуса отежава развој и евалуацију *NLP* модела, како класичних статистичких метода, тако и савремених дубоких неуронских мрежа [6].

Корпус је структурисана збирка писаних или говорних језичких података у машински читљивом облику, која се користи за анализу и обраду језика [6]. Величина и квалитет доступних корпуса значајно утичу на могућности развоја и поузданост *NLP* система за дати језик. Овај проблем је посебно изражен код језика са ограниченим ресурсима, као што је српски.

2.1 Историјски разлози за ограничене језичке ресурсе

Мали обим доступних дигиталних ресурса за српски језик је посљедица вишевијековног низа историјских околности које су директно утицале на очување писаног наслеђа. Српски је индоевропски језик који је развио богату средњовјековну књижевну традицију, укључујући темељна дјела попут Мирослављевог јеванђеља (око 1180.), Вукановог јеванђеља (око 1202.), Светосавског номоканона (1219.) и Душановог законика (1349.). Међутим, османска окупација (средина 15. – почетак 19. вијека) разорила је ову традицију. Манастирске библиотеке, централне за производњу рукописа, суочиле су се са уништавањем и пљачком, при чему су рукописи спаљивани, крадени или продавани колекционарима у Русији, Аустрији или Венецији¹ [6].

Након ослобођења од османске окупације (1804–1815), Србија је постепено обнављала своју књижевну и културну инфраструктуру, али је њено текстуално наслеђе наставило да трпи ударце. Балкански ратови (1912–1913) и Први свјетски рат изазвали су значајна разарања, док је најкатастрофалнији губитак услиједио 1941. године, када су њемачке бомбе уништиле Народну библиотеку Србије, уништивши преко 500.000 томова, укључујући 1.424 средњовјековна ћирилична рукописа [6]. Ови губици су дугорочно осиромашили српске језичке и књижевне ресурсе, што данас отежава развој језичких технологија. Недостатак података додатно се продубљује ограниченом дигитализацијом, рестриктивним ауторским правима и недовољном институционалном подршком [6].

2.2 Писмо, дијалекти и фонолошке варијанте

Још један историјски развој са савременим значајем јесте језичка модернизација српског језика током постосманског културног препорода. Српски филолог и етнолог Вук Караџић предводио је свеобухватну језичку реформу почетком 19. вијека, утичући не само на стандардизацију српског језика већ и на стандар-

¹На примјер, Вуканово јеванђеље данас се налази у Руској националној библиотеци, док је Минхенски српски псалтир однесен у Баварску 1688. године.

дизацију хрватског језика. Ово је утицало на будућност српскохрватског језика и ширег јужнословенског језичког простора, при чему је Бечки књижевни договор из 1850. године, који су потписали српски, хрватски и словеначки интелектуалци, прогласио заједничким језиком са два писма, ћирилицом и латиницом [6].

Поред писма, српски посједује и двије фонолошке варијанте изговора старог словенског гласа јат (ѣ): екавску (нпр. „дете“, „лепо“) и ијекавску (нпр. „дијете“, „лијепо“). Додатно, српски језик обухвата пет дијалеката (призренско-тимочки, косовско-ресавски, шумадијско-војвођански, зетско-јужносанџачки и источнохерцеговачки), те носи препознатљиве морфосинтаксичке особине, попут богате падежне флексије и слободног реда ријечи у реченици [6].

Разлике између дијалеката могу довести до различитих облика истих ријечи у тексту. То је посебно релевантно за призренско-тимочки дијалекат јужне Србије, у којем се, услед историјских језичких контаката на простору Балканског језичког савеза, испољавају морфосинтаксичке варијације повезане са утицајем сусједних језика, укључујући бугарски и македонски. У односу на стандардни српски, нестандартне варијанте призренско-тимочног дијалекта могу показивати различите површинске облике, што додатно отежава њихову аутоматску обраду и претрагу [9]. Уколико би се међу говорницима у корпусу нашли и говорници оваквих регионалних варијетета, прилагођени анализатор развијен за стандардни српски (поглавље 6) не би нужно исправно нормализовао такве облике. Ова могућност није посебно испитивана у оквиру овог рада, већ се наводи као теоријски, а не емпиријски потврђен изазов. На примјер, у призренско-тимочким говорима јавља се облик „работим“, који се разликује од стандардног српског облика „радим“, иако има исто значење. Овакве разлике могу отежати претрагу текста уколико систем није прилагођен дијалекатским варијантама.

Варијабилност писма имала је директан утицај на имплементацију система описаног у овом раду. И претрага у *Elasticsearch*-у (поглавље 6) и обрада текста прије векторског уграђивања (поглавље 7) захтијевале су нормализацију писма, односно превођење садржаја у јединствени облик (латиницу) прије индексирања. На тај начин се исти појам, без обзира на то да ли је написан ћирилицом или латиницом, третира као један термин, а не као два различита облика.

Латинично писмо српског језика користи дијакритичке знакове, односно додатне ознаке којима се основно слово разликује од другог слова. У српској латиници карактеристична су слова *č, ć, š, ž* и *đ*, док ћирилица за исте гласове користи посебна слова *ч, ћ, ш, ж* и *ђ*. Ова разлика је значајна за претрагу јер исти појам може бити записан са дијакритичким знаковима или без њих.

Због тога је нормализација дијакритичких варијанти укључена у оба дијела система. У оквиру лексичке претраге (поглавље 6), обрада дијакритичких варијанти реализована је у оквиру прилагођеног анализатора. У припреми текста за векторско уграђивање (поглавље 7), нормализација дијакритика представљала је један од првих корака обраде транскрипта. На тај начин смањује се број различитих начина записивања истог појма и олакшава његово препознавање у претрази.

2.3 Морфолошка сложеност српског језика

Морфологија представља дио граматике који се бави врстама ријечи, њиховом структуром и промјеном њихових облика [10]. У српском језику посебно су значајне промјенљиве врсте ријечи, код којих се облик мијења у зависности од различитих граматичких категорија. Међу њима се издвајају именице, придјиви и глаголи, чија морфолошка својства доприносе великој варијабилности површинског облика текста.

Именице имају граматичке категорије рода, броја и падежа [11]. Њихова промјена по падежима назива се деклинација, при чему једна лексема може имати више различитих облика. Лексема је основна самостална јединица лексичког система која обухвата све граматичке облике и различита значења која једна ријеч може остварити [12]. Српски језик има седам падежа: номинатив, генитив, датив, акузатив, вокатив, инструмен-тал и локатив. На примјер, именица град може се појавити у облицима град, града, граду, град, граде,

градом, граду. Иако се површински облици разликују, они представљају различите облике исте лексеме. Именице припадају једном од три граматичка рода: мушком, женском или средњем [11].

Придјеви се са именицом уз коју стоје слажу у роду, броју и падежу. Због тога и они мијењају свој облик у зависности од граматичког контекста. На примјер, придјев млад јавља се у облицима млад човјек, млада жена, младо дијете, а кроз падеже у конструкцији млад човјек, младог човјека, младом човјеку, младим човјеком. Осим тога, код описних придјева постоји и могућност степеновања, нпр. висок, виши, највиши, што представља додатни извор морфолошке варијабилности [11].

Глаголи имају сложенији систем морфолошких категорија. Њихови облици зависе од лица и броја, времена, начина и глаголског вида, а српски језик има и различите неличне глаголске облике. Тако се глагол радити може појавити као радим, радиш, ради, радимо, радите, раде, док се кроз различита времена и облике јављају, на примјер, радио је, радиће, радио бих, радећи. Посебну категорију представљају глаголски придјеви, укључујући радни (радио, радила, радило) и трпни (урађен, урађена, урађено). Глаголе такође карактерише глаголски вид, при чему се разликују свршени и несвршени глаголи. Примјер свшеног облика је написати, док несвршеног би био писати [11].

Оваква морфолошка разноврсност значи да се иста лексема у тексту може појавити у великом броју различитих површинских облика. То представља важан изазов за аутоматску обраду и претрагу текста на српском језику, јер облик који се појављује у корисничком упиту не мора бити идентичан облику исте лексеме који се налази у тексту.

Дио ове варијабилности ријешен је у оквиру лексичке претраге описане у поглављу 6, гдје прилагођени анализатор, уз стематизацију (енг. *serbian_stemming*), своди различите облике исте лексеме на заједнички стем. Стематизација представља поступак свођења различитих облика ријечи на њихову заједничку основу, поменути стем, који се добија уклањањем одређених наставака и дијелова ријечи и не мора представљати стварну ријеч у језику. На тај начин се, на примјер, различити падежни облици исте именице могу повезати у претрази. Ипак, стематизација не може повезати ријечи које имају сродно значење, али различиту основу, као што су писати и написати. Оне представљају различите лексеме, па их стематизација не може повезати само на основу њихове семантичке сличности. Управо тај преостали дио варијабилности представља један од разлога за увођење семантичке, односно векторске претраге, описане у поглављу 7.

2.4 Вишезначност и семантичка сложеност

Поред морфолошке флексије, додатан проблем представља чињеница да се текстови на природним језицима често одликују великим степеном комплексности, као и нејасноћама и двосмисленостима, односно вишезначностима у изражавању [7]. Исти концепт може бити изражен различитим ријечима или формулацијама, што представља посебан изазов за лексичку претрагу. На примјер, корисник може претраживати појам „додаци исхрани“, док се у транскрипту подкаста говори о „суплементима“. Иако се ове формулације разликују на нивоу ријечи, њихово значење је блиско. Лексичка претрага заснована на подударану термина такву везу не може поуздано препознати, осим ако су одговарајуће ријечи унапријед наведене као синоними. Ово ограничење представља један од главних разлога за увођење семантичке, односно векторске претраге (поглавље 7), која омогућава поређење текстова на основу њиховог значења, а не само на основу подударану конкретних ријечи.

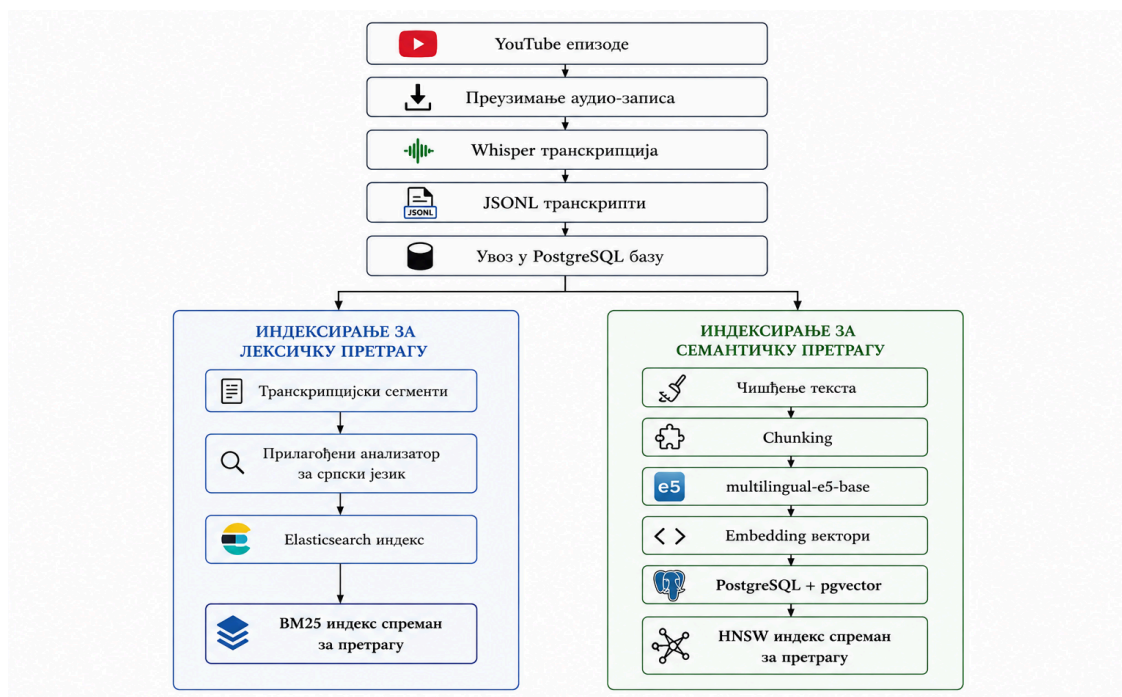
У овом поглављу дат је преглед цјелокупног приступа проблему и начина на који су појединачне компоненте система повезане у цјелину, док су имплементациони детаљи представљени у наредним главама рада. Систем се може посматрати кроз два основна тока: припрему података и индексирање, те обраду корисничког упита.

3.1 Припрема података и индексирање

Слика 1 приказује ток података од изворног *YouTube* садржаја до структура припремљених за лексичку и семантичку претрагу. Аудио-садржај преузет са *YouTube* канала пролази кроз транскрипцију (поглавље 5.2), након чега се добијени транскрипти чувају у *JSONL* формату (поглавље 5.3) и увозе у релациону базу података (поглавље 5.4). База затим представља централни извор структурираних података о каналима, епизодама и сегментима транскрипта (поглавље 5.5).

Након увоза података извршавају се два одвојена поступка индексирања: индексирање за лексичку претрагу (глава 6) и индексирање за семантичку претрагу (глава 7). У оквиру лексичке претраге, транскрипцијски сегменти пролазе кроз прилагођени анализатор за српски језик (поглавље 6.9), након чега се групно индексирају у *Elasticsearch*-у (поглавље 6.10). На тај начин формира се индекс над којим се релевантност резултата одређује примјеном алгорита *BM25* (поглавље 6.5).

Код семантичке претраге, транскрипцијски сегменти пролазе кроз чишћење (поглавље 7.10.1) и груписање у *chunk*-ове (поглавље 7.8.2), након чега се помоћу изабраног *embedding* модела (поглавље 7.6) претварају у *embedding* векторе. Вектори се чувају у релационој бази помоћу *pgvector* екстензије (поглавље 4.1), а над њима се формира *HNSW* индекс за приближну векторску претрагу (поглавље 7.9). На крају ове фазе формиране су двије структуре за претраживање, над којима се након пријема корисничког упита извршавају лексичка и семантичка претрага приказане у наредном одјељку.



Слика 1: Преглед припреме података и фаза индексирања

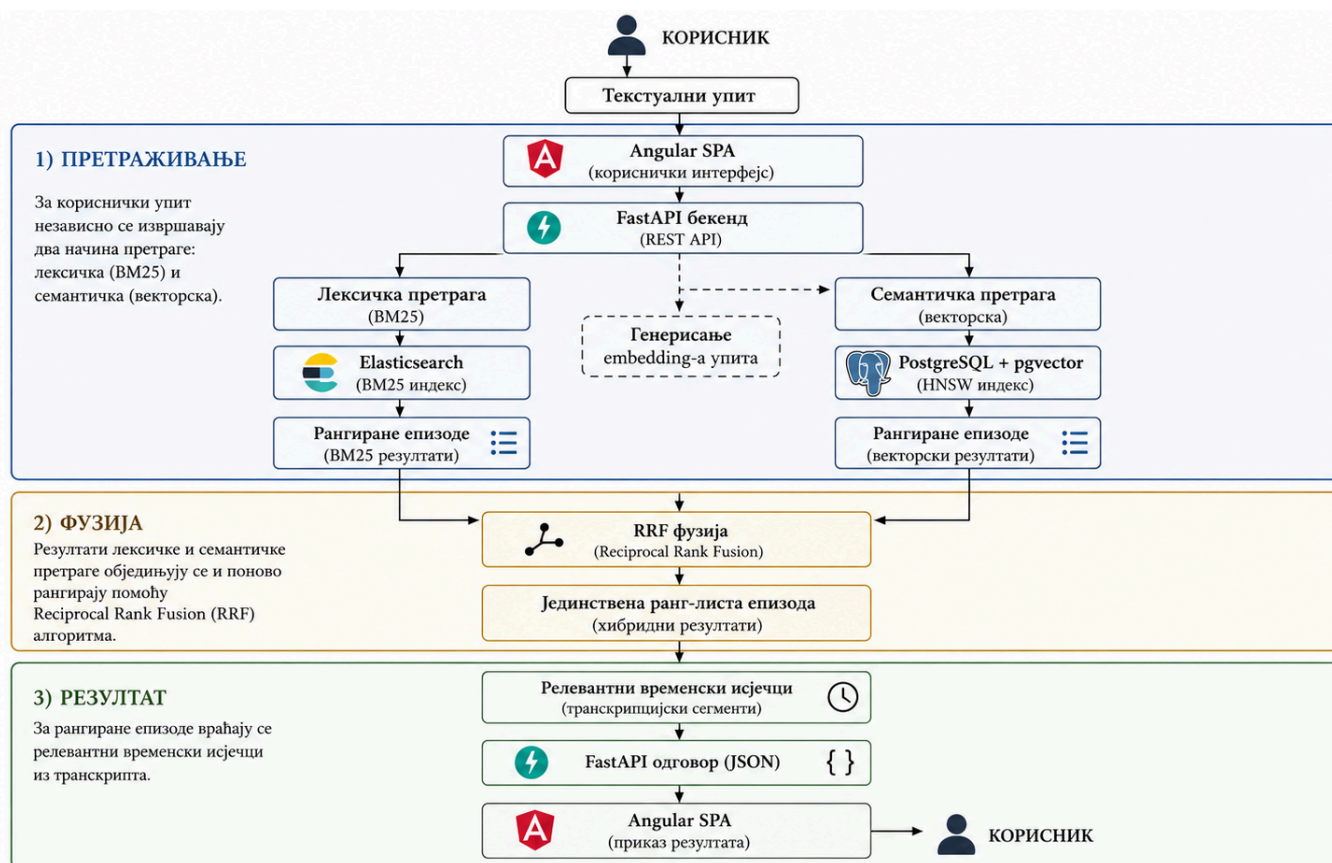
3.2 Обрада корисничког упита

Слика 2 приказује ток система који се покреће при свакој корисничкој претрази. Текстуални упит унесен кроз *Angular* кориснички интерфејс (поглавље 4.4) просљеђује се *FastAPI* позадинском сервису (поглавље 4.3), након чега се паралелно обрађује лексичком и семантичком претрагом.

У оквиру лексичке претраге, упит се просљеђује *Elasticsearch*-у, који претражује претходно формиран индекс и враћа ранг-листу епизода на основу лексичке релевантности (поглавље 6.11). За семантичку претрагу, кориснички упит се најприје претвара у *embedding* вектор помоћу истог *embedding* модела који је коришћен приликом индексирања, уз употребу префикса *query*. Затим се над *HNSW* индексом проналазе најсличнији *chunk*-ови и формира ранг-листа епизода (поглавље 7.11).

Добијене ранг-листе епизода обједињују се методом *Reciprocal Rank Fusion (RRF)*, описаној у глави 8. На основу позиције епизоде у обје листе израчунава се њен коначни *RRF* скор и формира јединствена хибридна ранг-листа (поглавље 8.5).

За рангиране епизоде прикупљају се и релевантни временски исјечци транскрипта пронађени лексичком и семантичком претрагом. Исјечци се обједињују и хронолошки сортирају, након чега *FastAPI* враћа резултате корисничком интерфејсу у јединственом формату.



Слика 2: Преглед обраде корисничког упита и формирања хибридних резултата

Систем описан у овом раду је реализован као скуп независних сервиса, међусобно повезаних преко заједничке мреже и покренутих помоћу *Docker Compose*-а. *Docker Compose* представља алат за дефинисање и покретање апликација састављених од више контејнера [13]. Контејнеризација омогућава да сваки сервис буде изолован у сопственом окружењу, са тачно дефинисаним зависностима, док се цијели систем може покренути и зауставити на јединствен начин.

Систем чини пет сервиса повезаних заједничком мрежом (*app-network*): база података (*PostgreSQL* са *pgvector* екстензијом), административни алат за базу (*pgAdmin 4*), претраживачки систем (*Elasticsearch*), позадински сервис (*FastAPI*) и кориснички интерфејс (*Angular*).

4.1 База података

PostgreSQL представља централну базу података и главни извор структурираних података у систему (енг. *source of truth*). У њој се чувају метаподаци о каналима и епизодама, као и сегменти транскрипта. Поред тога, помоћу екстензије *pgvector*, у бази се чувају векторске репрезентације текста (енг. *embeddings*), које се користе за семантичку претрагу описану у поглављу 7. *pgvector* је екстензија отвореног кода која *PostgreSQL*-у додаје подршку за чување вектора и претрагу према њиховој сличности [14]. Разлози за избор *pgvector*-а у односу на друга рјешења, као што су *FAISS*, *Qdrant*, *Milvus* и *Pinecone*, детаљније су образложени у поглављу 7.

За покретање базе користи се слика ***pgvector/pgvector:pg16***, заснована на *PostgreSQL*-у 16 и припремљена за коришћење *pgvector* екстензије. Сама екстензија се у бази креира кроз *Alembic* миграцију. Подаци се чувају у именованом *Docker* волумену, чиме се обезбјеђује њихово чување и након поновног покретања контејнера. Спремност сервиса провјерава се помоћу *healthcheck*-а, који користи команду *pg_isready*. Конфигурација *PostgreSQL* сервиса приказана је у листингу 1.

```
postgres:
  image: pgvector/pgvector:pg16
  environment:
    - POSTGRES_DB=podcast_search
    - POSTGRES_USER=podcast_user
    - POSTGRES_PASSWORD=podcast_password
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U podcast_user"]
```

Листинг 1: Конфигурација *PostgreSQL* базе са *pgvector* екстензијом

За административни приступ бази података, укључујући преглед табела и ручно извршавање упита, коришћен је *pgAdmin 4*. Његовом веб интерфејсу приступа се преко порта 5050.

4.2 Претраживачки систем

Elasticsearch сервис је заснован на прилагођеној *Docker* слици изграђеној на службеној слици ***elasticsearch:8.13.2***. У слику је додат ***analysis-icu*** додаток [15], који омогућава напреднију *Unicode* нормализацију и обраду текстуалних знакова. Његова примјена у прилагођеном анализатору детаљније је описана у поглављу 6. Прилагођена слика дефинисана је сљедећим *Dockerfile*-ом, приказаним у листингу 2.

```
FROM docker.elastic.co/elasticsearch/elasticsearch:8.13.2
RUN bin/elasticsearch-plugin install --batch analysis-icu
```

Листинг 2: *Dockerfile* за изградњу прилагођене *Elasticsearch* слике са *analysis-icu* додатком

Сервис ради у режиму *single-node*, што значи да се *Elasticsearch* извршава као један чвор, без повезивања са другим чворовима у кластеру. *X-Pack* безбједносни механизми [16], који обезбјеђују аутентификацију и контролу приступа су искључени јер је систем коришћен у развојном и интерном окружењу. Спремност сервиса провјерава се помоћу *healthcheck*-а, који чека да статус кластера више не буде *red*. Комплетна конфигурација индекса и анализатора, као и поступци индексирања и претраге, описани су у поглављу 6.

4.3 Позадински сервис

Позадински дио система реализован је у програмском језику *Python 3.11*, уз коришћење оквира *FastAPI* и сервера *Uvicorn*. *Uvicorn* имплементира *ASGI* (енг. *Asynchronous Server Gateway Interface*) стандард и користи се за покретање апликације и обраду *HTTP* захтјева. *FastAPI* је коришћен за реализацију програмског интерфејса апликације (*API*) [17]. *Python* је у систему коришћен и за генерисање векторских репрезентација текста помоћу библиотеке *sentence-transformers*.

За рад са базом података користи се *SQLAlchemy* као *ORM* (енг. *Object-Relational Mapping*), уз *psycopg2-binary* као *PostgreSQL* управљачки програм. Промјенама структуре базе управља се помоћу *Alembic*-а, док се за рад са векторским колонама користи *Python* подршка за *pgvector*. Валидација конфигурације и података реализована је помоћу библиотека *Pydantic* и *pydantic-settings*, док се вриједности из окружења учитавају помоћу библиотеке *python-dotenv*. Комуникација са *Elasticsearch*-ом остварује се преко службеног *Python* клијента.

Docker слика позадинског сервиса гради се у једној фази, уз коришћење кеширања слојева. Зависности наведене у датотеци *requirements.txt* инсталирају се прије копирања изворног кода. На тај начин се приликом измјене кода може поново искористити постојећи слој са већ инсталираним зависностима, без њихове поновне инсталације. Модели преузети преко платформе *Hugging Face*, укључујући модел за генерисање векторских репрезентација, чувају се у именованом *Docker* волумену (*hf_cache*). Тиме се избјегава њихово поновно преузимање након поновног покретања или изградње контејнера.

4.4 Кориснички интерфејс

Кориснички интерфејс је реализован у *Angular*-у [18], а за његову изградњу користи се *Docker multi-stage build* [19], који раздваја фазу изградње апликације од фазе њеног покретања. У првој фази, заснованој на слици *node:22-alpine*, гради се продукцијска верзија апликације, док се у другој фази добијене статичке датотеке копирају у минималну *nginx:alpine* слику. На тај начин коначна слика садржи само датотеке неопходне за покретање апликације, без изворног кода и алата потребних само током изградње, чиме се смањује њена величина.

Angular апликација реализована је као *SPA* (енг. *Single Page Application*), што значи да се рутирање између страница обавља на страни клијента. Ово је посебно важно за директан приступ конкретној епизоди и одговарајућем тренутку у транскрипту. Због тога је *nginx* подешен тако да захтјеве за путање које не одговарају постојећим статичким датотекама просљеђује апликацији преко *index.html*, као што је приказано у листингу 3.

```
location / {
    try_files $uri $uri/ /index.html;
}
```

Листинг 3: Конфигурација *nginx*-а за просљеђивање захтјева *Angular SPA* апликацији

Овакво подешавање омогућава исправан директан приступ одређеној рути и освјежавање странице, јер *nginx* захтјев просљеђује *Angular*-овом механизму за рутирање умјесто да врати грешку 404. Додатно, *nginx* је подешен за компресију одговора помоћу *gzip*-а и кеширање статичких ресурса, као што су *JavaScript* и

CSS датотеке, чиме се смањује количина података која се преноси приликом поновног учитавања апликације.

4.5 Мрежа и покретање система

Сви сервиси комуницирају преко заједничке *Docker bridge* мреже (*app-network*), што омогућава међусобну комуникацију преко имена сервиса. На примјер, позадински сервис приступа бази преко имена хоста *postgres*, а не *localhost*. На тај начин није потребно излагати све интерне портове сервиса хост систему.

Основне команде за покретање и заустављање система, као и за уклањање именованих волумена, приказане су у листингу 4.

```
# Покретање свих сервиса у позадини уз поновну изградњу слика
docker compose up -d --build

# Заустављање и уклањање контејнера уз задржавање података у волуменима
docker compose down

# Заустављање и уклањање контејнера и именованих волумена
docker compose down -v
```

Листинг 4: Команде за покретање и заустављање *Docker Compose* сервиса

Глава 5

Транскрипција и припрема корпуса

Ово поглавље описује поступак прикупљања и припреме корпуса који представља основу за претрагу описану у наредним поглављима. Најприје је представљен обим и извор прикупљених података, а затим поступак транскрипције видео-садржаја. Након тога описани су начин организације и складиштења добијених података на диску, као и њихов увоз у релациону базу података која се користи у систему.

5.1 Обим корпуса

Корпус коришћен у овом раду прикупљен је са 32 *YouTube* канала на српском језику, који обухватају различите тематске области и формате, укључујући интервјуе, друштвено-политичке подкасте, едукативни и научно-популарни садржај, као и забавне формате. Укупно је прикупљено и транскрибовано 15.889 епизода.

Избор канала обухвата различите теме, говорнике и стилове комуникације, чиме се обезбјеђује хетерогеност корпуса. На тај начин корпус омогућава испитивање система над различитим врстама садржаја, умјесто у оквиру само једне тематске области. Табела 1 приказује преглед свих канала обухваћених корпусом, заједно са бројем епизода прикупљених са сваког канала.

Табела 1: Списак канала у корпусу и број транскрибованих епизода по каналу.

Канал	Број епизода	Канал	Број епизода
Без Цензуре	3.345	РТС Образовно-научни програм	2.105
РТС Око	1.725	Анђеласт	1.073
Појачало	701	Приче У Нама	701
РТС ТВ серије	828	РТС Ординација	789
Да сам ја неко	656	Никола Радин Подкаст	501
Иван Косогор Подкаст	353	РТС Квадратура круга	353
Катена мунди	340	Бизнис Приче	324
Нови Стандард	289	Јао Миле	244
Још подкаст један	234	Центар за антиауторитарне студије	157
Рубикон	119	Тампон зона	119
Вредно приче	115	Миљанов корнер	102
Сто минута буке	168	Два и по психијатра	90
Лукавство ума	85	Изокренута учионица	76
Рекетирање	75	Сада и овде	72
Другачији фитнес	49	Контранапад	46
Залет иновација	29	Ђао Невена	26

5.2 Прикупљање и транскрипција

Видео-садржај преузет са *YouTube* канала било је потребно претворити у текстуални облик погодан за даљу обраду. У ту сврху коришћена је технологија аутоматског препознавања говора (енг. *Automatic Speech Recognition, ASR*), која омогућава претварање говорног сигнала у текстуални облик. Један од савремених система заснованих на овој технологији је *Whisper*, модел за аутоматско препознавање и транскрипцију

говора који је развила компанија *OpenAI*. Модел је обучен на великом вишејезичном скупу аудио-података и намијењен је аутоматској транскрипцији говорног садржаја, уз подршку за различите језике и облике говора [20].

Наведене карактеристике биле су значајне за избор *Whisper*-а у овом раду, будући да корпус садржи говор различитих говорника, са варијацијама у изговору, акценту и интонацији, као и могућим присуством позадинске буке.

5.3 Формат складиштења

Резултат транскрипције за сваку епизоду складиштен је у *JSONL* (енг. *JSON Lines*), текстуалном формату у којем сваки ред датотеке представља један самосталан и ваљан *JSON* објекат, за разлику од класичног *JSON* формата, у којем цијела датотека представља један објекат или низ објеката.

Овакав приступ омогућава инкрементално читање и обраду датотеке ред по ред, без потребе да се цјелокупна датотека одједном учита у меморију. То је посебно значајно у овом случају, будући да један ред, односно једна епизода, садржи цјелокупан транскрипт са временски означеним сегментима, па датотека са хиљадама епизода једног канала може достићи знатну величину.

Подаци су организовани на нивоу канала: један канал одговара једној *.jsonl* датотеци, а сваки ред једној епизоди тог канала. Датотеке се додатно компресују помоћу *gzip*-а (*.gz* екстензија), чиме се смањују потребан простор на диску и количина података приликом преноса.

У листингу 5 дат је скраћени приказ структуре једног реда, односно једне транскрибоване епизоде. Поља *description* и *plain_text*, као и низ *timestamped_segments* са низом *words*, скраћени су ради прегледности. Пуна структура садржи цјелокупан транскрипт и временске ознаке за појединачне ријечи.

```
{
  "video_id": "upxT3PGinY0",
  "title": "Branislav Ristivojević - Kosovo i Metohija je ogledalo srpske politike / Rubikon podcast 105",
  "description": "Гост емисије Рубикон био је... [скраћено]",
  "published_at": "2025-06-12T17:01:10Z",
  "duration": "PT1H4M28S",
  "view_count": "58228",
  "like_count": "443",
  "channel_title": "Rubikon",
  "channel_id": "UCb0hTyq2vHyxaimv5cgjAdw",
  "tags": [],
  "file_path": "/var/youtube/rubikon/upxT3PGinY0.webm",
  "download_time": "2025-08-28T20:15:34.711178",
  "url": "https://www.youtube.com/watch?v=upxT3PGinY0",
  "transcription": {
    "plain_text": "Nama nešto opasno je u redu, nama trebaju neki novi filteri... [скраћено]",
    "timestamped_segments": [
      {
        "id": 0,
        "start": 0.0,
        "end": 2.58,
        "text": "Nama nešto opasno je u redu, nama trebaju neki novi filteri.",
        "words": [
          {
            "word": " Nama",
            "start": 0.0,
            "end": 0.16,
            "probability": 0.93
          }
        ]
      }
    ]
  }
}
```

Листинг 5: Примјер структуре JSON записа са метаподацима и транскрипцијом епизоде

Као што се види, један ред садржи метаподатке епизоде (наслов, опис, интернет адресу, датум објаве, број прегледа и свиђања), као и цјелокупан резултат транскрипције сачињеног из текста транскрипта (*plain_text*) и низа временски означених сегмената (*timestamped_segments*). *Whisper* у изворном излазу пружа и детаљнији ниво грануларности, на примјер за сваки сегмент доступан је низ појединачних ријечи, при чему свака ријеч има сопствени временски опсег и вјероватноћу препознавања (*probability*). Међутим, овај ниво детаља намјерно се не преноси у базу података приликом увоза. Задржавају се само временски опсег (*start, end*) и текст цјелокупног сегмента.

Оваква грануларност омогућава прецизно позиционирање корисника на одговарајући тренутак у видеу приликом претраге. На тај начин резултат претраге не упућује само на одговарајућу епизоду већ и на конкретан дио видеа у којем се тражени садржај појављује.

5.4 Складиштење у PostgreSQL бази

Подаци из *.jsonl.gz* датотека увозе се и трајно складиште у *PostgreSQL* релационој бази података. *PostgreSQL* представља извор истине (енг. *source of truth*) за структуриране податке о каналима, епизодама и сегментима транскрипта. Детаљи о самом сервису, контејнеризацији и екстензији *pgvector*, која се касније користи за векторску претрагу (поглавље 7), обрађени су у поглављу 4.

5.5 Модел података

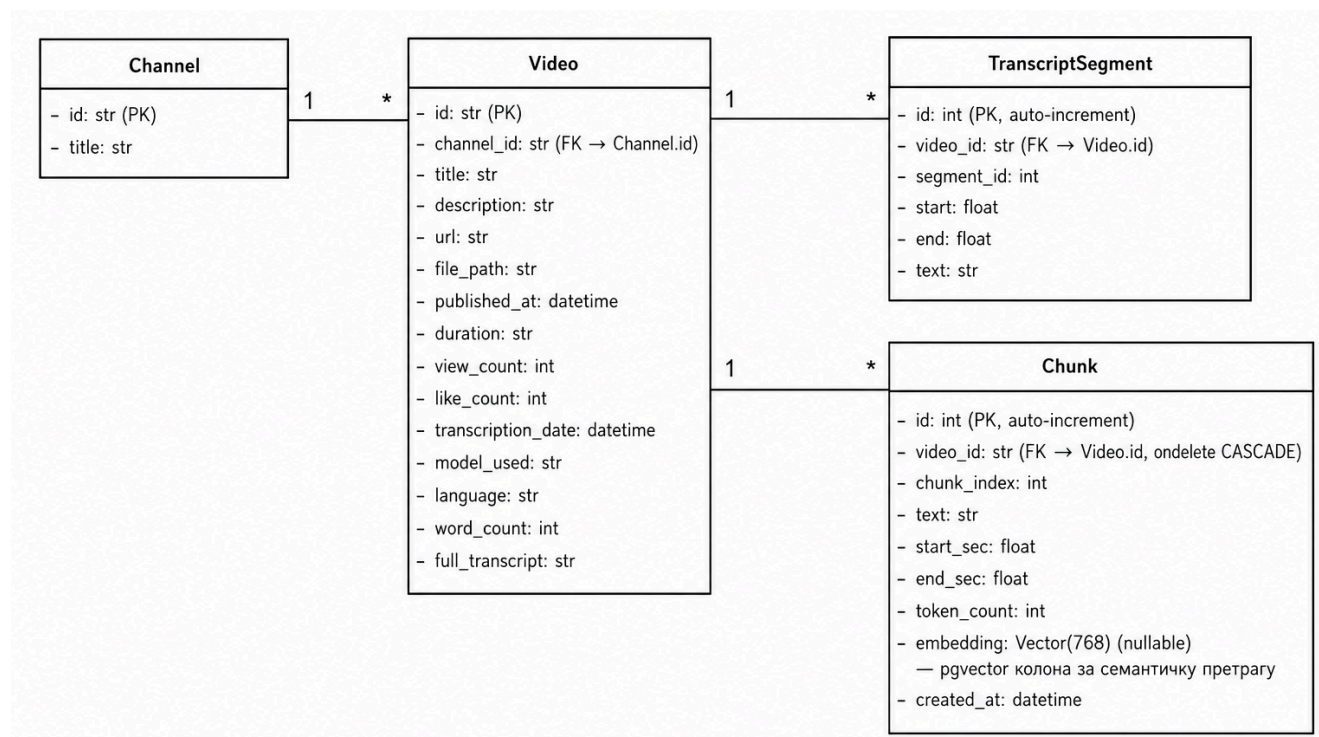
Подаци у бази организовани су кроз четири повезана ентитета. Три ентитета одговарају структури изворних *.jsonl.gz* датотека, док се четврти генерише накнадно за потребе семантичке претраге:

- Канал (*Channel*) представља један запис за сваки *YouTube* канал;
- Видео-епизода (*Video*) представља један запис за сваку транскрибовану епизоду, повезан са тачно једним каналом;
- Сегмент транскрипта (*TranscriptSegment*) представља један запис за сваки временски означени дио транскрипта, повезан са тачно једном епизодом;
- *Chunk* представља један запис за сваку текстуалну цјелину насталу груписањем узастопних сегмената транскрипта, намијењену генерисању векторских уграђивања (енг. *embedding*) за семантичку претрагу, што је детаљније објашњено у поглављу 7.

Другим ријечима, структура података на диску односно једна *.jsonl.gz* датотека по каналу, један ред по епизоди и више сегмената унутар транскрипта се директно пресликава у релациони модел. Један канал има више епизода (1:N), док једна епизода има више сегмената транскрипта (1:N) и више *chunk*-ова (1:N).

Табела *Chunk* садржи и колону *embedding* типа *Vector(768)* (*pgvector* тип, поглавље 4), као и јединствени индекс над паром *video_id* и *chunk_index*, који обезбјеђује јединственост редног броја *chunk*-а унутар једне епизоде.

Дијаграм класа на слици 3 приказује сва четири ентитета, њихове кључне атрибуте и везе између њих.

Слика 3: Дијаграм класа модела података: *Channel*, *Video*, *TranscriptSegment* и *Chunk*.

5.6 Функција за увоз података у базу

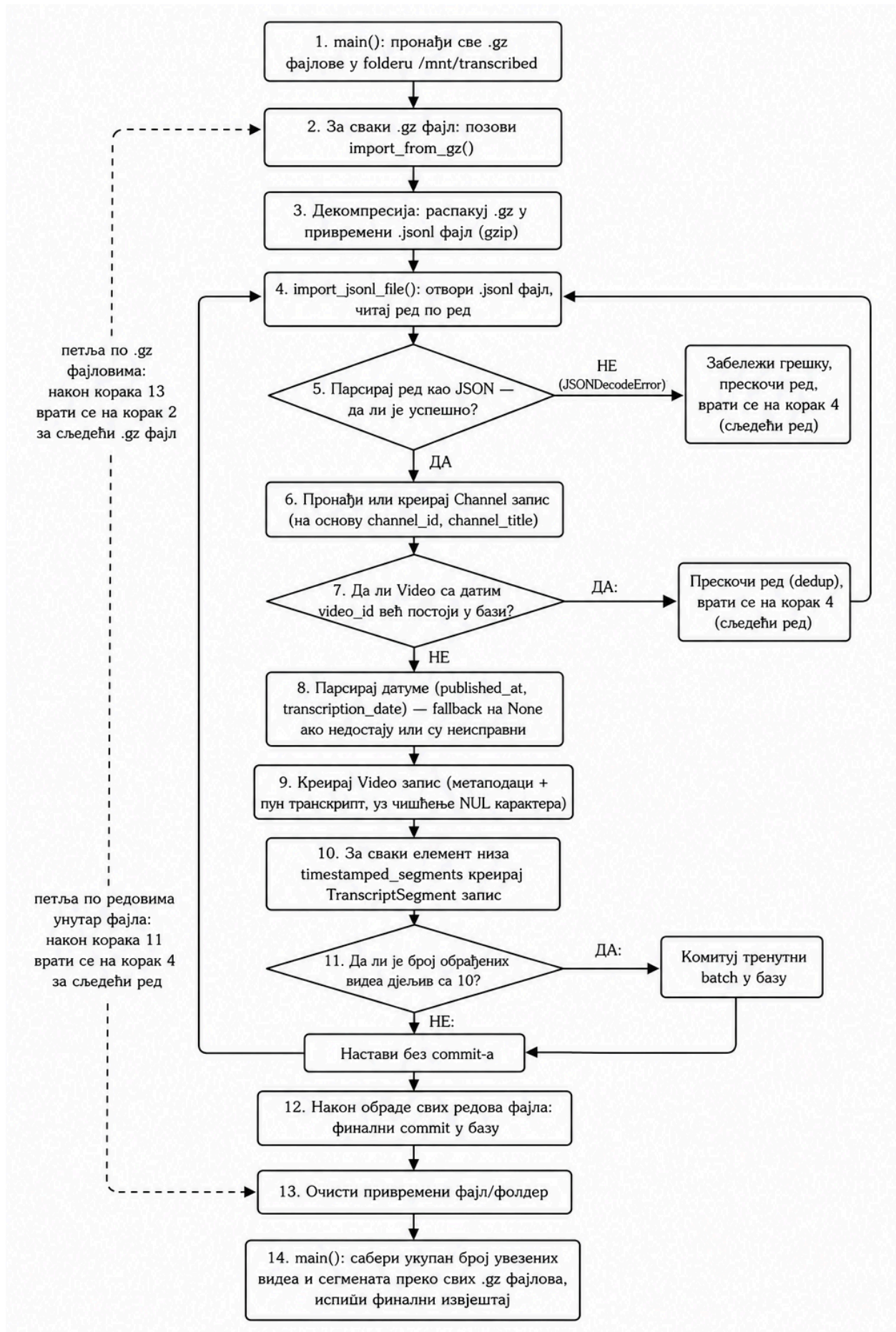
Увоз података из *.jsonl.gz* датотека у *PostgreSQL* базу обавља се скриптом *import_data.py*, која се извршава једнократно или по потреби, приликом додавања нових или ажурираних датотека. Скрипта проналази све *.gz* датотеке у директоријуму */mnt/transcribed*, декомпресује их у привремене *.jsonl* датотеке и редом обрађује њихов садржај.

Током обраде провјеравају се исправност *JSON* записа и постојање видеа у бази. На тај начин неисправни редови могу бити прескочени, док поновни увоз истих података не доводи до дуплирања. За нове записе креирају се одговарајући ентитети *Channel*, *Video* и *TranscriptSegment*, а промјене се периодично потврђују у трансакцији ради ефикаснијег рада. Након обраде свих датотека привремене датотеке се уклањају, а скрипта формира завршни извјештај о броју увезених видеа и сегмената транскрипта.

Комплетан ток поступка приказан је на слици 4, а скрипта се покреће унутар *backend* контејнера командом приказаном у листингу 6.

```
docker compose exec backend python -m app.import_data
```

Листинг 6: Покретање скрипте за увоз података унутар *backend* контејнера

Слика 4: Алгоритам увоза података из *.jsonl.gz* датотека у PostgreSQL базу.

Глава 6

Лексичка претрага (*Elasticsearch*)

Лексичка претрага (енг. *lexical search*), позната и као претрага по кључним ријечима (енг. *keyword search*) или претрага пуног текста (енг. *full-text search*), представља приступ претраживању текста заснован на директном подударану термина из корисничког упита са терминима садржаним у документима. Лексичка претрага се ослања на статистичке методе као што су *TF-IDF* (енг. *Term Frequency-Inverse Document Frequency*) и његово унапређење *Okapi BM25* (енг. *Best Matching 25, BM25*). *TF-IDF* мјери важност одређене ријечи унутар документа у односу на цјелокупну колекцију докумената [21], док *BM25* рангира документе на основу учесталости термина упита у документу, ријеткости тог термина у корпусу и дужине документа у односу на просјечну дужину докумената у колекцији [22]. У оквиру овог рада, лексичка претрага је имплементирана помоћу система *Elasticsearch*.

6.1 Шта је *Elasticsearch*

Elasticsearch је систем намијењен брзом претраживању и анализи великих количина података. Омогућава претраживање различитих врста података, од текста до структурираних података, као и њихову анализу. Његова предност се огледа у могућности брзог и ефикасног претраживања великих количина информација, због чега се користи у различитим познатим системима. Википедија (енг. *Wikipedia*) га користи за претрагу текстуалног садржаја и давање приједлога корисницима. *The Guardian* га користи за праћење реакција публике на нове чланке, док *Stack Overflow* помоћу њега проналази слична питања и одговоре. Платформа *GitHub* га користи за претраживање велике количине програмског кода [23].

6.2 *Elasticsearch* наспрам релационе базе података

Релационе базе података, као што је *PostgreSQL*, првенствено су намијењене поузданом чувању и обради структурираних података, као и извршавању прецизно дефинисаних упита. Иако *PostgreSQL* подржава претрагу пуног текста, таква претрага није његова примарна намјена и захтијева додатну конфигурацију и одговарајуће индексе. Код система чији је основни задатак претрага великих количина текстуалног садржаја јавља се потреба за напреднијом анализом текста и ефикасним рангирањем резултата.

За разлику од релационих база података, *Elasticsearch* је специјализован за претрагу и анализу текста. Он омогућава индексирање текстуалних докумената, њихову лексичку анализу и рангирање резултата према релевантности за упит. Поред тога, подржава механизме који омогућавају флексибилнију претрагу, попут толерисања типографских грешака и обраде различитих облика ријечи. Због тога је *Elasticsearch* погодан за дио система који је задужен за претрагу великог корпуса текстуалних докумената, док релациона база задржава улогу чувања и управљања структурираним подацима.

6.3 Дистрибуирана архитектура

Elasticsearch је дистрибуиран систем који се састоји од једног или више чворова (енг. *nodes*) повезаних у кластер (енг. *cluster*). Чвор представља покренуту инстанцу *Elasticsearch*-а, док кластер представља групу чворова који заједнички управљају подацима и ресурсима система. Чворови који припадају истом кластеру се идентификују помоћу заједничког назива кластера (*cluster.name*). *Elasticsearch* може да функционише и са само једним чвором, при чему тај чвор истовремено представља цијели кластер [23].

Фрагмент (енг. *shard*) представља физичку јединицу у којој *Elasticsearch* складишти дио података једног индекса. Индекс је логичка цјелина која може бити састављена од једног или више фрагмената, док се документи физички чувају и индексирају унутар њих. Сваки фрагмент представља засебну инстанцу претраживачког механизма и може да се посматра као самостална јединица за складиштење и претрагу [23].

Копија (енг. *replica*) представља копију примарног фрагмента и служи као додатни ниво заштите података у случају отказа хардвера или недоступности чвора на којем се налази примарни фрагмент. Поред обезбјеђивања редундантности, копија може да учествује и у обради захтјева за читање, као што су претрага и преузимање докумената, чиме се може повећати капацитет система за операције читања [23].

Број примарних фрагмената и њихових копија се одређује у конфигурацији индекса. У случају постављања *Elasticsearch*-а у дистрибуирано окружење, на примјер коришћењем *Kubernetes* кластера са више чворова, *Elasticsearch* чворови и фрагменти могу да се распореде на више физичких или виртуелних машина. На тај начин различити фрагменти могу да буду смјештени на различитим чворовима, а упит који обухвата више фрагмената може да се обрађује паралелно. Оваква архитектура омогућава хоризонтално скалирање система и расподјелу оптерећења на више рачунарских ресурса, што може да побољша перформансе претраге при великим количинама података и већем броју истовремених захтјева. У оквиру овог рада *Elasticsearch* се извршава локално на једном чвору, што је довољно за потребе имплементираног система.

6.4 Како *Elasticsearch* чува и претражује податке?

Један од основних концепата на којима се заснива ефикасна претрага текста у *Elasticsearch*-у јесте инвертовани индекс (енг. *inverted index*) [23]. За разлику од приступа у којем се полази од документа и у њему траже одговарајуће ријечи, инвертовани индекс успоставља обрнуту везу: за сваки термин чува информацију о документима у којима се тај термин појављује [24]. Ова структура може да се упоређи са индексом на крају књиге, гдје се уз одређену ријеч наводе странице на којима се она појављује. На сличан начин, *Elasticsearch* за сваки термин чува податке о документима који га садрже.

Када корисник пошаље упит, *Elasticsearch* не мора да пролази кроз сваки документ појединачно. Умјесто тога, претрага се врши над већ изграђеним индексом, чиме се значајно смањује количина података коју је потребно обрадити. Поред информације о документима у којима се термин појављује, индекс може да садржи и додатне информације, попут учесталости термина и његове позиције у документу. Ови подаци се користе приликом претраге и одређивања релевантности резултата [24].

6.5 *BM25* и рангирање резултата

Проналажење докумената који садрже термине из корисничког упита представља само први корак лексичке претраге. Уколико више докумената садржи исте термине, потребно је одредити њихову релевантност и поређати их тако да се најрелевантнији резултати прикажу на почетку. У *Elasticsearch*-у се за ту намјену користи алгоритам *Okapi BM25* (енг. *Best Matching 25, BM25*), који представља подразумевани модел за израчунавање оцјене релевантности и рангирање докумената [25].

У имплементираном систему *BM25* представља основу за рангирање резултата лексичке претраге. Основни текстуални упит је реализован помоћу *match* упита над пољем *text*, након чега *Elasticsearch* сваком пронађеном документу додјељује одговарајућу оцјену релевантности. На тај начин се резултати не враћају само на основу постојања подударана са упитом, већ се међусобно рангирају према степену релевантности.

У оквиру упита се додатно користе услови *must* и *filter*. Услови наведени у *must* дијелу представљају обавезна подударања која учествују у израчунавању оцјене релевантности, док се *filter* користи за ограничавање скупа докумената без утицаја на ту оцјену. Оваква подјела омогућава раздвајање критеријума који утичу на релевантност од услова који служе искључиво за филтрирање резултата.

BM25 у овом раду није имплементиран као посебан алгоритам, већ се користи као уграђени механизам за рангирање који обезбјеђује *Elasticsearch*. На тај начин лексичка претрага користи инвертовани индекс за ефикасно проналажење докумената који одговарају упиту, док *BM25* одређује њихов релативни поредак према релевантности.

6.6 Ниво индексирања

Једна од првих одлука приликом имплементације лексичке претраге се односила на избор нивоа на којем ће се подаци индексирати. Разматране су три могућности:

- **Један документ по транскрипцији видеа** што би значило један документ представља цјелокупну транскрипцију једног видеа;
- **Један документ по сегменту** што би значило сваки транскрипцијски сегмент представља засебан документ;
- **Један документ по транскрипцији видеа са угњежденим сегментима** што би значило сегменти се чувају унутар документа који представља цијели видео.

Иако би индексирање на нивоу видеа било једноставније и омогућило би директно рангирање резултата по видеу, такав приступ не би омогућио једноставно враћање прецизног временског тренутка у којем се тражени садржај појављује. Са друге стране, чување сегмената као засебних докумената омогућава да сваки резултат претраге садржи текст сегмента и његову временску ознаку. На тај начин се кориснику може приказати конкретан дио разговора и омогућити директан прелазак на одговарајући тренутак у видеу.

Због тога је у овом раду изабрана **грануларност на нивоу транскрипцијског сегмента**, односно сваки сегмент представља један документ у *Elasticsearch* индексу. Ова одлука је значајна и за каснију имплементацију семантичке претраге, јер се и она заснива на мањим текстуалним јединицама изведеним из транскрипцијских сегмената. На тај начин оба приступа претрази раде над мањим дијеловима транскрипта, што олакшава њихово касније повезивање у хибридној претрази.

Трећа могућност, чување транскрипције једног видеа као документа са угњежденим сегментима, није изабрана јер би увела додатну сложеност у структуру индекса и упите за претрагу, а није била неопходна за остваривање захтјева система. Индексирање сваког сегмента као засебног документа је омогућило једноставније претраживање и директно повезивање пронађеног текста са његовом временском ознаком.

6.7 Мапирање

У *Elasticsearch*-у се структура података дефинише помоћу мапирања (енг. *mapping*), које одређује тип и начин индексирања појединачних поља документа. Мапирање се може упоредити са шемом у релационим базама података, гдје структура табеле дефинише типове и карактеристике њених колона. На основу мапирања *Elasticsearch* одређује како ће се вриједности појединих поља чувати, анализирати и индексирати, што директно утиче на начин на који се над тим пољима извршава претрага [23].

Приликом дефинисања мапирања *Elasticsearch* индекса било је потребно одредити која поља треба индексирати и који тип података им одговара. Текст транскрипцијских сегмената је дефинисан као поље типа `text`, јер представља примарни садржај над којим се врши претрага пуног текста (енг. *full-text search*). Насупрот томе, идентификатори и поља која се користе за филтрирање, сортирање или груписање су дефинисани одговарајућим типовима података. На тај начин је мапирање прилагођено стварним потребама претраге, без копирања цјелокупне структуре података која већ постоји у релационој бази.

Приликом избора поља је узета у обзир и улога релационе базе као примарног извора података. Поља која нису неопходна за претрагу, као што су опис епизоде, трајање, број прегледа, број свиђања, путања до датотеке и интернет адреса (енг. *Uniform Resource Locator, URL*), нису дуплирана у *Elasticsearch*-у. Опис видеа није укључен у садржај над којим се врши претрага јер представља метаподатак, а не транскрибовани садржај епизоде. Вриједности попут броја прегледа и броја свиђања се, поред тога, могу мијењати током времена, па се њиховим чувањем само у *PostgreSQL*-у избјегава непотребна синхронизација између два система. Након претраге *Elasticsearch* враћа идентификаторе релевантних видеа, а остали подаци се на основу њих дохватају из релационе базе ради приказа на корисничком интерфејсу.

У *Elasticsearch* су укључена поља која су директно повезана са претрагом, филтрирањем и приказом резултата: идентификатор видеа (`video_id`), идентификатор сегмента (`segment_id`), садржај сегмента (`text`),

наслов видеа (`title`), временски почетак сегмента (`start`), временски крај сегмента (`end`), наслов канала (`channel_title`) и вријеме објаве видеа (`published_at`).

Поље `text` је дефинисано као тип `text` и представља основно поље за претрагу транскрипта. Наслов видеа (`title`) је такође дефинисан као `text`, чиме је омогућена претрага наслова епизода. Идентификатор видеа (`video_id`) је дефинисан као `keyword`, јер се користи за идентификацију видеа, груписање резултата и њихово повезивање са подацима у релационој бази. Идентификатор сегмента (`segment_id`) служи за идентификацију конкретног транскрипцијског сегмента, док `start` и `end` одређују његов временски интервал унутар видеа. Поље `channel_title` омогућава филтрирање по каналу, док се `published_at` користи за рад са датумом објављивања.

На тај начин *Elasticsearch* индекс садржи само податке који су потребни за претрагу, филтрирање, груписање и идентификацију резултата, док *PostgreSQL* остаје централно мјесто за трајно чување комплетних података о епизодама. Оваква подјела одговорности смањује непотребно дуплирање података и омогућава да се промјенљиви подаци ажурирају у релационој бази без потребе за њиховим поновним индексирањем у *Elasticsearch*-у.

Мапирање се поставља у конфигурацији индекса. На тај начин *Elasticsearch* добија информацију о структури докумената и начину на који треба да обрађује њихова поља приликом индексирања и претраге. Дио конфигурације којим је дефинисано мапирање коришћено у овој имплементацији је приказан у листингу 7.

```
"mappings": {
  "properties": {
    "video_id": {
      "type": "keyword"
    },
    "segment_id": {
      "type": "integer"
    },
    "text": {
      "type": "text",
      "analyzer": "serbian_analyzer",
      "search_analyzer": "serbian_analyzer"
    },
    "title": {
      "type": "text",
      "analyzer": "serbian_analyzer",
      "search_analyzer": "serbian_analyzer"
    },
    "start": {
      "type": "float"
    },
    "end": {
      "type": "float"
    },
    "channel_title": {
      "type": "keyword"
    },
    "published_at": {
      "type": "date"
    }
  }
}
```

Листинг 7: Конфигурација мапирања *Elasticsearch* индекса

6.8 Изазови српског језика у *Elasticsearch*-у

Примјена *Elasticsearch*-а над корпусом српског језика доноси неколико језички специфичних изазова који су детаљније обрађени у поглављу 2:

- богата флексија, односно постојање различитих граматичких облика исте ријечи, као што су „Косово“, „Косову“, „Косова“ и „Косовом“;
- паралелна употреба ћирилице и латинице;
- употреба слова са дијакритичким знаковима и различитих начина њиховог записивања.

Стандардни анализатор *Elasticsearch*-а не узима у обзир све ове специфичности српског језика. *Elasticsearch* нуди поједине компоненте за обраду српског текста, укључујући зауставне ријечи и стематизацију, али је за потребе овог система било неопходно прилагодити њихово комбиновање и додати кораке потребне за уједначавање различитих начина записивања текста. Због тога је дефинисан прилагођени анализатор (енг. *analyzer*), који је описан у наставку поглавља.

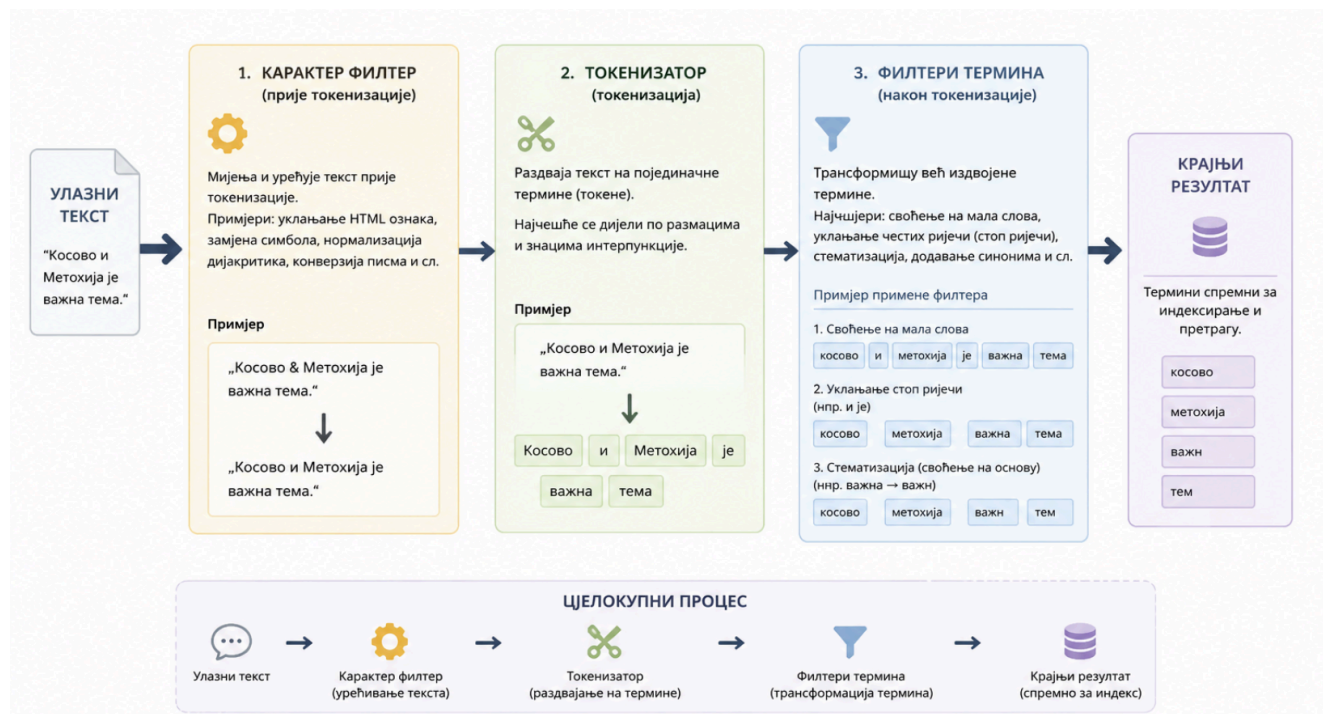
6.9 Анализатор

Анализатор (енг. *analyzer*) представља ланац трансформација кроз који текст пролази прије него што се запише у инвертовани индекс, током индексирања, или прије него што се упоређи са садржајем индекса током претраге [23].

Сваки анализатор се састоји од три основне компоненте:

- карактер филтера (енг. *character filter*) који мијења текст прије токенизације;
- токенизатора (енг. *tokenizer*) који раздваја текст на појединачне термине;
- низа филтера термина (енг. *token filter*) који трансформишу већ издвојене термине, на примјер свођењем на мала слова, уклањањем честих ријечи или стематизацијом [23].

На слици 5 је приказан поједностављен ток анализе текста кроз наведене компоненте анализатора.



Слика 5: Поједностављен приказ процеса анализе текста у *Elasticsearch*-у

Elasticsearch подразумевано нуди стандардни (енг. *standard*) анализатор, који представља опште рјешење за анализу текста. Он раздваја текст на термине према границама ријечи, уклања већину знакова интерпункције и претвара термине у мала слова. Међутим, такав приступ не узима у обзир језичке специфичности

као што су морфологија, зауставне ријечи (енг. *stop words*) и различити облици исте ријечи. Због тога стандардни анализатор није био довољан за претрагу корпуса на српском језику, па је за потребе овог система дефинисан прилагођени ланац филтера.

6.9.1 Прилагођени анализатор за српски језик

За потребе лексичке претраге је коришћен прилагођени анализатор за српски језик. *Elasticsearch*, поред стандардног анализатора, нуди и компоненте прилагођене српском језику, укључујући нормализацију текста, уклањање зауставних ријечи и стематизацију. Њиховим комбиновањем са додатним филтерима је формиран ланац анализе прилагођен потребама овог система [26].

Текст најприје пролази кроз филтер **lowercase**, којим се сви термини свде на мала слова. На тај начин се избегава разликовање истих ријечи само на основу величине слова. Након тога се примјењује **icu_folding** филтер из *ICU (International Components for Unicode)* додатка. Његова улога је уједначавање различитих *Unicode* варијанти знакова и додатна обрада дијакритичких разлика [27].

Сљедећи корак представља **serbian_normalization**, односно нормализација прилагођена српском језику. Овим филтером се ћирилички и латинички записи свде на заједнички облик, чиме се омогућава да различити начини записивања исте ријечи буду третирано као исти термин. На примјер, „Србија“ и „Srbija“ након анализе могу да буду сведени на исти облик. Овај корак је посебно значајан због паралелне употребе ћирилице и латинице у српском језику.

Послије нормализације се примјењује филтер за уклањање зауставних ријечи (енг. *stop words*). У имплементацији је дефинисан филтер **serbian_stop**, који користи уграђену листу српских зауставних ријечи **_serbian_** [28]. Његова примјена је посебно корисна код транскрипата, у којима се често појављују кратке функционалне ријечи које не доприносе значајно релевантности резултата. Примјер зауставних ријечи је приказан у листингу 8 [29].

```
"ili", "a", "ali", "pa", "biti", "ne", "jesam", "sam", "jesi", "si",
"je", "jesmo", "smo", "jeste", "ste", "jesu", "su", "nijasam", "nisam", "nijas",
"nisi", "nije", "nijasmo", "nismo", "nijeste", "niste", "nijas", "nisu", "budem", "budes",
"bude", "budemo", "budete", "budu", "budes", "bih", "bi", "bismo", "biste", "biše",
"bise", "bio", "bili", "budimo", "budite", "bila", "bilo", "bile", "ću", "ćeš",
"će", "ćemo", "ćete", "neću", "nećeš", "neće", "nećemo", "nećete", "cu", "ces",
"ce", "cemo", "cete", "necu", "neces", "nece", "necemo", "necete", "mogu",
"možeš", "može", "možemo", "možete", "mozes", "moze", "mozemo", "mozete", "и", "или",
"a", "али", "па", "бити", "не", "јесам", "сам", "јеси", "си", "је",
"јесмо", "смо", "јесте", "сте", "јесу", "су", "нијесам", "нисам", "нијеси",
"ниси", "није", "нијесмо", "нисмо", "нијесте", "нисте", "нијесу", "нису", "будем", "будеш",
"буде", "будемо", "будете", "буду", "будес", "бих", "би", "бисмо", "бисте",
"бише", "бисе", "био", "били", "будимо", "будите", "била", "било", "биле", "ћу",
"ћеш", "ће", "ћемо", "ћете", "нећу", "нећеш", "неће", "нећемо", "нећете", "цу",
"цес", "це", "цемо", "цете", "нецу", "нецес", "неце", "нецемо", "нецете", "могу",
"можеш", "може", "можемо", "можете", "мозес", "мозе", "моземо", "мозете"
```

Листинг 8: Примјер зауставних ријечи из уграђене листе за српски језик

За рјешавање морфолошке варијативности српског језика примјењује се стематизација. Филтер **serbian_stemming** користи **stemmer** са параметром **language: "serbian"** [30]. Његова улога је свођење различитих граматичких облика ријечи на заједничку основу, чиме се повећава вјероватноћа да ће упит написан у једном облику пронаћи и документе у којима се појављује други облик исте ријечи. На примјер, претрага термина „Косово“ може да обухвати и појављивања облика „Косову“ или „Косова“.

Конечан редослијед филтера индексног анализатора је: *lowercase*, *icu_folding*, *serbian_normalization*, *serbian_stop* и *serbian_stemming*. Конфигурација анализатора који се користи приликом индексирања поља *text* и *title* је приказана у листингу 9.

```

"analysis": {
  "analyzer": {
    "serbian_analyzer": {
      "type": "custom",
      "tokenizer": "standard",
      "filter": [
        "lowercase",
        "icu_folding",
        "serbian_normalization",
        "serbian_stop",
        "serbian_stemming"
      ]
    }
  },
  "filter": {
    "serbian_stop": {
      "type": "stop",
      "stopwords": "_serbian_"
    },
    "serbian_stemming": {
      "type": "stemmer",
      "language": "serbian"
    }
  }
}

```

Листинг 9: Конфигурација анализатора за индексирање текста на српском језику

6.9.2 Скраћенице: анализатор за претрагу

Анализатор који се користи приликом индексирања не рјешава проблем скраћеница и акронима. На примјер, корисник који унесе „КиМ“ може очекивати резултате у којима се појављује пуни назив „Косово и Метохија“, иако ова два записа не представљају исти низ термина. Нормализација и стематизација саме по себи не могу да успоставе овакву везу. Због тога се приликом претраге користи додатни филтер за синониме. Његова улога је да дефинисане скраћенице повеже са њиховим пуним називима и другим еквивалентним облицима.

Синоними се у овом систему примјењују само приликом претраге, а не приликом индексирања. На тај начин промјена листе скраћеница не захтијева поновно индексирање цјелокупног корпуса. Због тога су дефинисана два одвојена анализатора: **serbian_analyzer**, који се користи при индексирању, и **serbian_search_analyzer**, који се користи за обраду корисничког упита.

Претраживачки анализатор почиње истим корацима као индексни анализатор. Текст се најприје своди на мала слова помоћу филтера **lowercase**, након чега се примјењују **icu_folding** и **serbian_normalization**. Тек након нормализације се примјењује **serbian_synonyms**, тако да се синоними обрађују над већ уједначеним облицима термина. Послије проширивања скраћеница се уклањају зауставне ријечи помоћу филтера **serbian_stop**, а на крају се примјењује **serbian_stemming**.

Редослијед филтера у претраживачком анализатору је: *lowercase*, *icu_folding*, *serbian_normalization*, *serbian_synonyms*, *serbian_stop* и *serbian_stemming*. Конфигурација анализатора који се користи приликом претраге је приказана у листингу 10.

```
"analysis": {
  "filter": {
    "serbian_synonyms": {
      "type": "synonym",
      "synonyms": SYNONYMS
    }
  },
  "analyzer": {
    "serbian_search_analyzer": {
      "type": "custom",
      "tokenizer": "standard",
      "filter": [
        "lowercase",
        "icu_folding",
        "serbian_normalization",
        "serbian_synonyms",
        "serbian_stop",
        "serbian_stemming"
      ]
    }
  }
}
```

Листинг 10: Конфигурација анализатора за претрагу са подршком за синонине

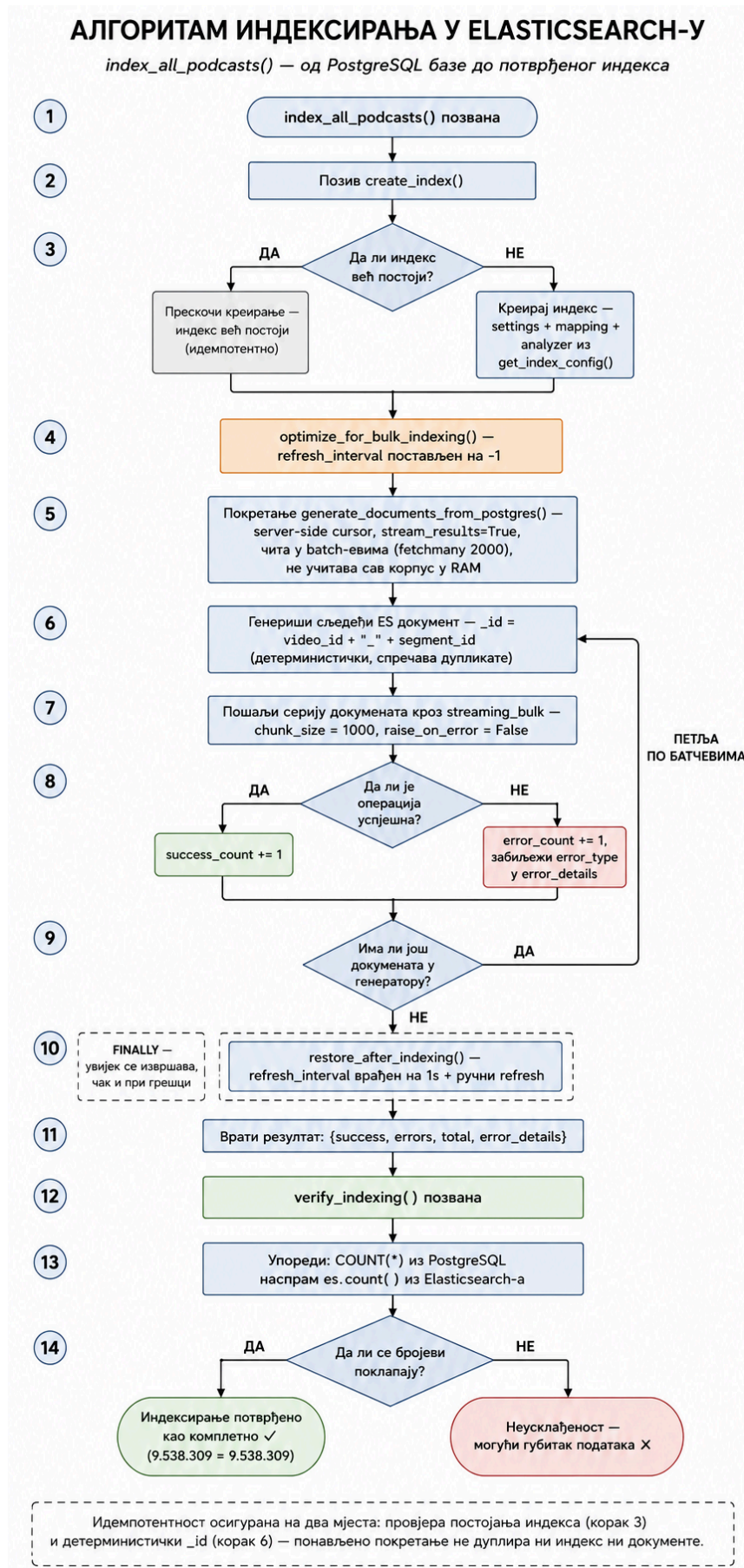
Листа скраћеница и њихових еквивалентних облика која се користи у претраживачком анализатору је приказана у листингу 11.

```
SYNONYMS = [
  "kim, kosovo i metohija, kosmet",
  "rs, republika srpska",
  "cg, crna gora, montenegro",
  "eu, evropska unija",
  "nato, sjeverno-atlantski savez",
  "un, ujedinjene nacije",
  "mmf, medjunarodni monetarni fond",
  "rts, radio televizija srbije",
  "sns, srpska napredna stranka",
  "sps, socijalisticka partija srbije",
  "bdp, bruto domaci proizvod",
]
```

Листинг 11: Списак синонима коришћених у *Elasticsearch* претрази

6.10 Алгоритам индексирања

Цјелокупан поступак индексирања, од провјере постојања индекса, преко генерисања и слања докумената, до завршне провјере, приказан је дијаграмом тока на слици 6. Поједини кораци су детаљније објашњени у наставку овог одјељка.



Слика 6: Алгоритам индексирања у *Elasticsearch*-у од провјере постојања индекса до провјере учитаних докумената

6.10.1 Креирање индекса

Функција `create_index()` прије креирања провјерава да ли индекс са датим именом већ постоји, што одговара кораку 3 на слици 6. На тај начин је операција идемпотентна, па поновљени позив функције, на примјер након поновног покретања позадинског сервиса, не доводи до брисања или поновног креирања постојећег индекса.

Уколико индекс не постоји, он се креира позивом *Elasticsearch API*-ја са конфигурацијом коју враћа функција `get_index_config()`. Конфигурација обухвата подешавања индекса, мапирање поља и анализатор за српски језик описан у претходним одјељцима.

6.10.2 Bulk индексирање

Индексирање преко девет и по милиона докумената захтијева другачији приступ од слања посебног захтјева за сваки документ. За ту намјену се користи *Bulk API*, који омогућава да се више операција над документима пошаље у оквиру једног *HTTP* захтјева. На тај начин се смањује број појединачних мрежних захтјева и убрзава поступак индексирања [31]. Кораци овог поступка су приказани на слици 6.

6.10.2.1 Зашто *bulk*, а не документ по документ

Када би се за сваки документ слао засебан *HTTP* захтјев, трошак комуникације и обраде захтјева би се понављао више од девет и по милиона пута. Такав приступ би значајно успорио почетно попуњавање индекса.

Bulk API омогућава да један захтјев садржи више операција над документима, чиме се исти фиксни трошак распоређује на читаву групу докумената [31].

Да би се цјелокупан корпус прослиједио *Bulk API*-ју без учитавања свих докумената истовремено у меморију, документи се генеришу један по један помоћу функције `generate_documents_from_postgres()`, што одговара кораку 5 на слици 6.

Идентификатор сваког документа се експлицитно поставља у облику `f"{video_id}_{segment_id}"`, као што је приказано у кораку 6. На тај начин исти транскрипцијски сегмент приликом поновног индексирања добија исти `_id`. Ако документ са тим идентификатором већ постоји у индексу, његов садржај се замјењује новим умјесто да се креира дупликат.

6.10.2.2 Оптимизација и покретање

Функција `optimize_for_bulk_indexing()` прије почетка масовног индексирања привремено искључује освјежавање индекса тако што поставља `refresh_interval` на `-1`, што одговара кораку 4 на слици 6. Током почетног попуњавања индекса није неопходно да сваки нови документ одмах постане доступан за претрагу, па се искључивањем периодичног освјежавања смањује непотребно трошење ресурса.

Након завршетка индексирања се `refresh_interval` враћа на уобичајену вриједност. Овај корак се извршава у `finally` блоку, па се подешавање враћа и у случају да током индексирања дође до грешке. Овакав поступак је у складу са препорукама за убрзавање почетног индексирања великих количина података [32].

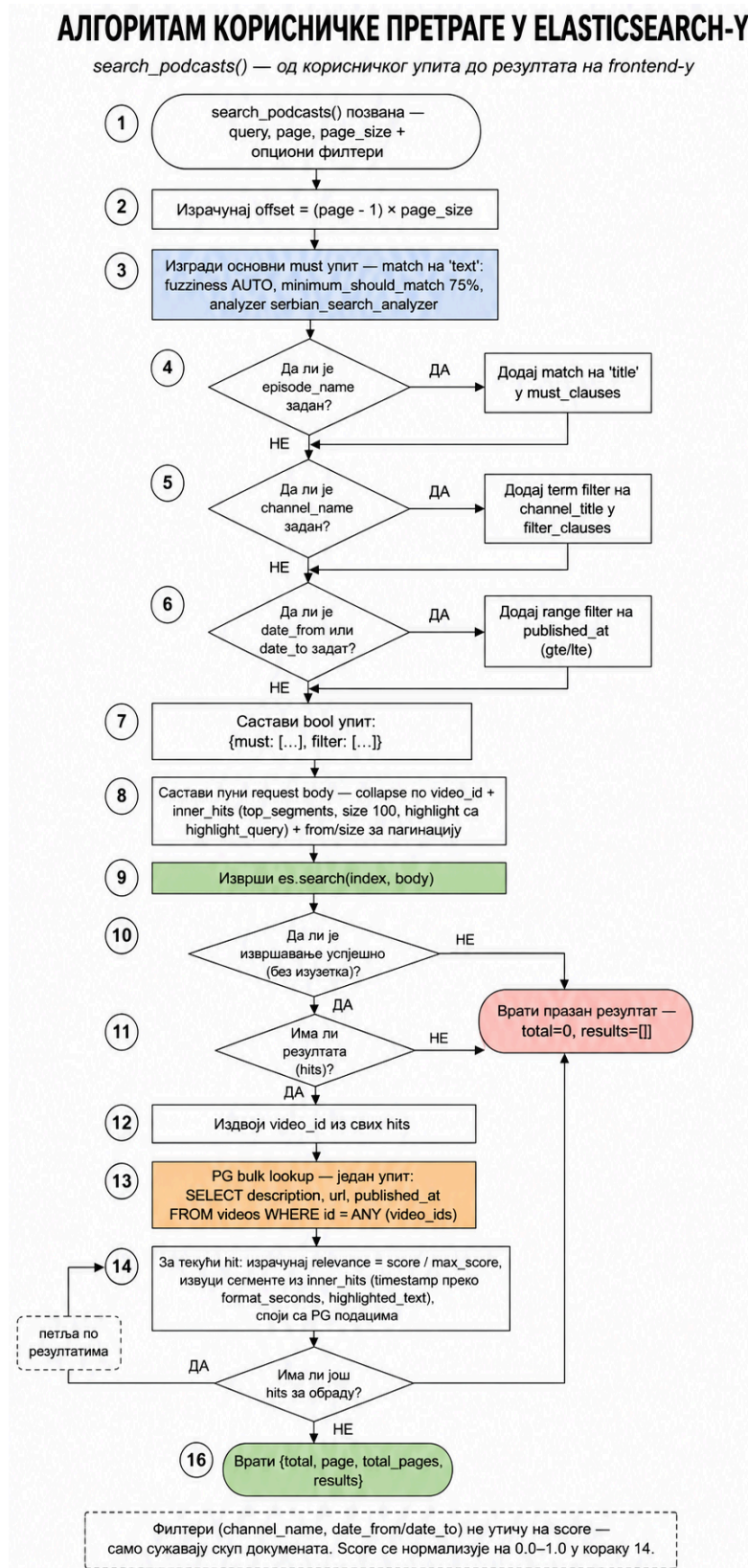
Поступак индексирања се покреће преко административне крајње тачке `POST /admin/index/create`, након што се провјери доступност базе и *Elasticsearch* кластера. Резултат индексирања цјелокупног корпуса је приказан у листингу 12.

```
{"success": 9538309, "errors": 0, "total": 9538309}
```

Листинг 12: Резултат *bulk* индексирања докумената

6.11 Алгоритам претраге

За разлику од индексирања, које се извршава приликом припреме индекса, поступак претраге се покреће за сваки кориснички упит. Он обухвата обраду примљеног упита, формирање *Elasticsearch* захтјева, извршавање претраге и припрему резултата за кориснички интерфејс. Цјелокупан ток је приказан на слици 7.

Слика 7: Алгоритам корисничке претраге у *Elasticsearch-у*

6.11.1 Грађење *Elasticsearch* упита

Функција `search_podcasts()` прима текстуални упит (енг. *query*) као обавезан параметар, као и опционе параметре за назив епизоде, назив канала, почетни и крајњи датум. Поред њих, функција прима и параметре страницења *page* и *page_size*, што одговара кораку 1 на слици 7.

Основни дио претраге представља **match** упит над пољем **text**. Параметар **fuzziness**: **"AUTO"** омогућава толерисање одређених разлика између термина упита и термина у индексу, што је корисно код мањих правописних грешака. Параметар **minimum_should_match**: **"75%"** одређује колики дио анализираних термина из упита мора да буде присутан у документу, док **analyzer**: **"serbian_search_analyzer"** задаје претраживачки анализатор описан у претходном одјелку. **match** упит представља један од основних упита за претрагу анализираних текстуалних поља у *Elasticsearch*-у [33].

Конфигурација основног упита је приказана у листингу 13.

```
must_clauses = [
    {
        "match": {
            "text": {
                "query":          query,
                "fuzziness":      "AUTO",
                "minimum_should_match": "75%",
                "analyzer":      "serbian_search_analyzer"
            }
        }
    }
]
```

Листинг 13: Конфигурација **match** упита за претрагу текста у *Elasticsearch*-у

Опциони услови се додају у упит само ако их је корисник задао, што је приказано у корацима 4, 5 и 6. Назив епизоде се додаје као додатни **match** услов у **must_clauses**, док се назив канала и временски опсег додају у **filter_clauses**. Услови у **must** дијелу учествују у израчунавању *BM25* оцјене релевантности, док услови у **filter** дијелу ограничавају скуп докумената без утицаја на њихов скор.

6.11.2 collapse, inner_hits и highlight

Прије слања захтјева се формира комплетно тијело *Elasticsearch* упита. За начин приказивања резултата су посебно важни механизми **collapse**, **inner_hits** и **highlight**.

collapse се примјењује над пољем **video_id**, чиме се резултати групишу тако да се сваки видео прикаже само једном. За сваки видео се у **inner_hits** издваја до 100 најрелевантнијих транскрипцијских сегмената. На тај начин један резултат представља епизоду, док су унутар њега доступна конкретна мјеста у транскрипту која одговарају упиту.

Конфигурација **highlight** механизма служи за означавање дијелова текста који одговарају упиту. Пронађени дијелови се означавају HTML ознаком **<mark>**, што омогућава њихово визуелно истицање на корисничком интерфејсу.

6.11.3 Извршавање, дохватање података из PostgreSQL-а и формирање одговора

Позив **es.search()**, приказан у кораку 9 на слици 7, налази се у **try/except** блоку. У случају грешке при извршавању упита функција враћа празан резултат (**total: 0, results: []**), умјесто да се грешка пренесе до корисничког интерфејса. Исти облик празног одговора се враћа и када је претрага успјешно извршена, али није пронађен ниједан одговарајући документ.

Ако резултати постоје, **video_id** вриједности враћених докумената се издвајају у јединствену листу. Након тога се једним упитом према *PostgreSQL* бази дохватају метаподаци за све пронађене видео-записе. На тај начин се избјегава слање посебног упита бази за сваки резултат. Исјечак кода је приказан у листингу 14.


```

video_ids = [hit["_source"]["video_id"] for hit in hits]

result = conn.execute(
    text("""
        SELECT id, description, url, published_at
        FROM videos
        WHERE id = ANY(:ids)
    """),
    {"ids": video_ids}
)
pg_data = {row.id: row for row in result}

```

Листинг 14: Дохватање метаподатака из релационе базе

За сваки резултат се затим израчунава нормализована оцјена релевантности. Сирова *Elasticsearch* оцјена се дијели максималном оцјеном у тренутном скупу резултата, а добијена вриједност се заокружује на двије децимале. Ова нормализација служи за једноставнији приказ релативне релевантности резултата у оквиру једног упита и не представља апсолутну мјеру релевантности.

Временске ознаке транскрипцијских сегмената се истовремено претварају из секунди у читљив формат помоћу функције **`format_seconds()`**. На примјер, вриједност **83.4** се приказује као **"1:23"**. Дио поступка је приказан у листингу 15.

```

relevance = round((hit["_score"] / max_score), 2)

matches = []
for seg_hit in hit["inner_hits"]["top_segments"]["hits"]["hits"]:
    seg = seg_hit["_source"]
    highlighted = seg_hit.get("highlight", {}).get(
        "text", [seg["text"]]
    )[0]

    matches.append({
        "timestamp": format_seconds(seg["start"]),
        "timestamp_seconds": int(seg["start"]),
        "text": seg["text"],
        "highlighted_text": highlighted
    })

```

Листинг 15: Нормализација оцјене релевантности и форматирање временских ознака сегмената

На крају се подаци добијени из *Elasticsearch*-а, као што су наслов, канал, релевантни сегменти и оцјена релевантности, спајају са метаподацима из *PostgreSQL* базе. Тако формиран резултат се враћа као одговор лексичке претраге, а користи се и као један од улаза у поступак хибридне претраге описан у поглављу 8.

Семантичка (векторска) претрага

Семантичка (енг. *semantic search*), односно векторска претрага (енг. *vector search*), представља приступ претраживању текста код којег се, умјесто директног поређења термина, пореди значење текстуалног садржаја [34]. Упит и садржај документа се представљају у облику нумеричких вектора, а њихова сличност се одређује на основу положаја тих вектора у вишедимензионалном простору. Текстови који изражавају слично значење могу имати блиске векторске репрезентације и када не дијеле исте ријечи. На тај начин семантичка претрага допуњује лексичку претрагу описану у поглављу 6. У оквиру овог рада, семантичка претрага је имплементирана коришћењем *embedding* модела за генерисање векторских репрезентација и *pgvector* екстензије *PostgreSQL* базе података за њихово складиштење и претрагу по сличности.

Потреба за оваквим приступом произлази из ограничења лексичке претраге, која су дјелимично описана у поглављу 2.4. Проблем није ограничен на појединачне, унапријед познате парове синонима, попут „додаци исхрани“ и „суплементи“, јер се такви случајеви могу ријешити ручно одржаваном листом синонима, слично начину на који су скраћенице попут „КиМ“ обрађене у *Elasticsearch*-у (поглавље 6.9.2). Међутим, проблем је шире природе и у литератури информационог претраживања је познат као вокабуларни јаз (енг. *vocabulary problem*). Једна од првих систематских емпиријских студија овог проблема је показала да двоје људи, независно једно од другог, за исти појам или радњу бирају исти термин у мање од 20% случајева [35]. У систему описаном у овом раду корисник формулише упит својим ријечима, док гост подкаста исти појам може изразити на другачији начин. Због тога лексичко подударање може изостати и када су упит и дио транскрипта тематски повезани. На примјер, упит „како узгајати медитеранску културу“ може бити повезан са дијелом транскрипта у којем се говори о томе да „маслине траже доста сунца“ и да је „смоква захвална биљка“, иако упит и транскрипт не садрже исте значајне термине.

Додатни проблем могу представљати грешке настале приликом аутоматске транскрипције. Уколико *ASR* модел, односно систем за аутоматско препознавање говора описан у поглављу 5.2, погрешно транскрибује термин „дречавци“, његов очекивани облик можда неће постојати у индексу. У том случају лексичка претрага може пропустити релевантан резултат без обзира на начин на који *BM25* рангира пронађене документе.

Лексичка претрага има ограничења и када исти термин има различита значења у зависности од контекста. На примјер, термин „култура“ може се појавити у садржају о медитеранској пољопривреди, култури сјећања или бактеријској култури. Само присуство истог термина није довољно да се одреди на које се од ових значења корисников упит односи. Семантичка репрезентација текста омогућава да се приликом поређења узме у обзир и шири контекст у којем се термин појављује.

7.1 Векторске репрезентације текста

Векторска репрезентација текста (енг. *embedding*) представља пресликавање текста у вектор у вишедимензионалном простору. Циљ оваквог представљања је да текстови сличног значења имају блиске векторске репрезентације. Према начину представљања, вектори могу бити ријетки (енг. *sparse*) и густе (енг. *dense*). Код ријетких репрезентација већина координата има вриједност нула, док су код густих репрезентација координате углавном различите од нуле. За разлику од лексичких репрезентација, које се заснивају на присуству и учесталости термина, научене *embedding* репрезентације омогућавају представљање семантичких односа између текстова у векторском простору [36].

У оквиру овог система су коришћене густе (*dense*) векторске репрезентације текста. Модел пресликава кориснички упит и дијелове транскрипта у векторе фиксне димензије, након чега се њихова сличност ко-

ристи за проналажење семантички релевантног садржаја. На тај начин систем може да препозна сличност и између текстова који не користе исте термине, али изражавају слично значење.

Кључно својство *embedding* репрезентација је да добијени вектор зависи од цјелокупног текста који се просљеђује моделу, а не само од појединачних термина [34]. Због тога различити контексти у којима се иста ријеч појављује могу довести до различитих векторских репрезентација текста. На примјер, текст о „медитеранској култури“ и текст о „култури сјећања“ могу бити смјештени у различите дијелове векторског простора, иако оба садрже ријеч „култура“. Модел приликом генерисања вектора узима у обзир и остале ријечи у тексту, чиме се омогућава прецизније представљање његовог значења.

Термин *embedding* се у литератури користи у два повезана значења [34]. Може да означава поступак пресликавања података у векторски простор, али и резултат тог поступка, односно конкретну векторску репрезентацију. У наставку овог рада се значење термина одређује на основу контекста у којем се користи.

7.2 Мјера сличности: *dot product*, *cosine similarity* и еуклидско растојање

Сваки вектор има два основна својства: дужину, односно норму ($\|A\| = \sqrt{\sum_i a_i^2}$), и смјер. Код текстуалних *embedding* вектора смјер код многих модела носи информацију о семантичкој сличности, док норма може бити повезана са другим својствима репрезентованог текста, као што су његова дужина или количина садржаја [34]. Ова разлика представља један од фактора који утичу на избор мјере сличности.

Један од начина поређења два вектора је унутрашњи производ (енг. *dot product*), чија геометријска интерпретација гласи:

$$q \cdot u = \|q\| \cdot \|u\| \cdot \cos(\theta)$$

Унутрашњи производ зависи и од угла θ између вектора и од производа њихових норми [36]. Због тога два вектора истог смјера, али различитих норми, не морају имати исту вриједност унутрашњег производа. У контексту претраге је ово важно када норма *embedding* вектора зависи од својстава улазног текста, јер тада на резултат поређења не утиче само њихов смјер [34].

Косинусна сличност (енг. *cosine similarity*) уклања утицај норме тако што унутрашњи производ дијели производом норми оба вектора:

$$\cos(q, u) = \frac{q \cdot u}{\|q\|_2 \|u\|_2}$$

На тај начин вриједност мјере зависи од угла између вектора, односно њиховог смјера [36]. Косинусна сличност има вриједности у интервалу $[-1, 1]$, при чему вриједност 1 означава исти смјер вектора, а 0 њихову ортогоналност. Код текстуалних *embedding* репрезентација већа косинусна сличност обично указује на већу семантичку сличност, у зависности од својстава конкретног модела [34].

Ово својство је посебно значајно у систему описаном у овом раду, јер се пореде текстови различите дужине. Кориснички упит може да садржи свега неколико ријечи, док дио транскрипта обухвата знатно више текста. Косинусна сличност омогућава да се утицај различитих норми уклони и да се поређење заснива на смјеру вектора.

Трећа често коришћена мјера је еуклидско растојање (енг. *Euclidean distance*, L_2), које представља удаљеност између два вектора у простору [36]:

$$d(q, u) = \sqrt{\sum_i (q_i - u_i)^2}$$

За разлику од претходне двије мјере, код којих већа вриједност означава већу сличност, код еуклидског растојања мања вриједност означава већу блискост вектора. Еуклидско растојање зависи и од њихових норми, што се може видјети из односа [36]:

$$\|q - u\|_2^2 = \|q\|_2^2 - 2(q \cdot u) + \|u\|_2^2$$

Због тога вектори сличног смјера, али различитих норми, могу имати различито еуклидско растојање. У систему, описаном у овом раду, као основна мјера изабрана је косинусна сличност, како би, приликом поређења нагласак био стављен на смјер *embedding* вектора.

7.3 Нормализација вектора и избор мјере сличности

Нормализација вектора подразумијева свођење његове норме на јединичну вриједност, при чему смјер вектора остаје непромијењен [34]. Када су два вектора нормализована, њихов унутрашњи производ је једнак косинусној сличности, јер важи $\|q\|_2 = \|u\|_2 = 1$ [34], [36]:

$$q \cdot u = \cos(q, u)$$

За нормализоване векторе се и еуклидско растојање може изразити преко косинусне сличности:

$$\|q - u\|_2^2 = 2 - 2 \cos(q, u)$$

Због тога косинусна сличност, унутрашњи производ и еуклидско растојање над нормализованим векторима дају исти поредак резултата, уз обрнут смјер рангирања код еуклидског растојања [36].

Ово својство се користи и у систему описаном у овом раду. *Embedding* вектори транскрипта се нормализују приликом њиховог генерисања, док се упитни вектор нормализује при свакој претрази. На тај начин се поређење заснива на смјеру вектора, а не на њиховој норми.

Овакав поступак одговара и начину употребе модела ***multilingual-e5-base*** који је коришћен у овом раду и детаљније описан у одјелку 7.6. Након нормализације вектора њихов унутрашњи производ одговара косинусној сличности, што омогућава да се семантичка сличност одређује на основу смјера векторских репрезентација.

7.4 Проблем приближне претраге и HNSW

Проналажење k вектора из колекције који су најсличнији датом упитном вектору назива се *top-k* претрагом [36]. Код мањих колекција је могуће упоредити упитни вектор са сваким вектором у колекцији и затим изабрати најсличније резултате. Међутим, са растом броја вектора овакав приступ постаје све скупљи, јер је за сваки упит потребно извршити поређење са цијелом колекцијом [36].

Због тога се код великих колекција користе алгоритми приближне претраге најближих сусједа (енг. *Approximate Nearest Neighbor*, ANN). За разлику од исцрпне претраге, ови алгоритми не пореде упит са сваким вектором, већ настоје да ефикасније пронађу најближе кандидате. Тиме се убрзава претрага, уз могућност да пронађени резултати не садрже увијек апсолутно најближе векторе.

За приближну претрагу у овом систему је коришћен *Hierarchical Navigable Small World (HNSW)* индекс, који подржава *pgvector*. *HNSW* организује векторе у облику вишеслојног графа, при чему везе између чворова омогућавају ефикасно кретање према векторима који су блиски упиту [37].

Претрага почиње на вишим слојевима графа, који садрже мањи број чворова и омогућавају брже кретање кроз простор. Након проналажења одговарајуће области, претрага се наставља на нижим слојевима, све до основног слоја који садржи све векторе. На тај начин није потребно поредити упитни вектор са сваким вектором у колекцији, што *HNSW* чини погодним за претрагу великих колекција *embedding* вектора [37].

7.5 pgvector као подршка за векторску претрагу

Складиштење и претрага *embedding* вектора у овом систему су реализовани помоћу *pgvector* екстензије за *PostgreSQL*, чија је основна конфигурација описана у поглављу 4.1. Приликом избора технологије су разматране и алтернативе као што су *FAISS*, *Qdrant*, *Milvus* и *Pinecone*.

FAISS представља библиотеку намијењену ефикасној претрази и груписању густих вектора, док *Qdrant* и *Milvus* представљају специјализована рјешења за рад са векторским подацима. *Pinecone* је управљана услуга за векторску претрагу у облаку. Увођење неког од ових рјешења би у архитектури овог система подразумевало додатну компоненту за складиштење или претрагу вектора.

За потребе овог рада је *pgvector* изабран јер омогућава да се *embedding* вектори чувају у истој *PostgreSQL* бази у којој се већ налазе остали подаци система. На тај начин није било потребно уводити засебну векторску базу у архитектуру, а векторске репрезентације су остале повезане са подацима о транскриптим на нивоу релационе базе.

Додатна предност *pgvector*-а је могућност комбиновања векторске претраге са стандардним *SQL* упитима и условима над подацима у *PostgreSQL*-у. У тренутној имплементацији ова могућност није директно искоришћена за филтрирање резултата семантичке претраге, али оставља простор за будуће проширење система.

7.6 Избор *embedding* модела

Квалитет семантичке претраге у великој мјери зависи од избора *embedding* модела, који текст пресликава у векторску репрезентацију. За потребе овог система су разматрана три модела:

- *multilingual-e5-base*;
- *multilingual-e5-large* [38];
- *bge-m3* [39].

bge-m3 подржава три начина представљања и претраживања: *dense*, *sparse* и *multi-vector* [39]. Пошто је лексичка претрага у овом систему већ реализована помоћу *Elasticsearch*-а, за семантичку компоненту је био довољан модел намијењен генерисању густих векторских репрезентација. Због тога додатне могућности модела *bge-m3* нису биле неопходне за изабрану архитектуру, па је предност дата моделима из *E5* породице.

Између модела *multilingual-e5-base* и *multilingual-e5-large* је направљен компромис између квалитета претраге и потребних рачунарских ресурса. На *MIRACL* скупу за вишејезичко претраживање, *multilingual-e5-base* остварује просјечни *nDCG@10* од 62,3, док *multilingual-e5-large* остварује 66,5 [38]. *nDCG* (енг. *Normalized Discounted Cumulative Gain*) представља мјеру квалитета рангирања резултата претраге и детаљније је описан у поглављу о евалуацији.

Иако већи модел остварује бољи резултат на наведеном скупу, његово извршавање захтијева и више рачунарских ресурса. С обзиром на то да је систем развијан и тестиран у локалном окружењу са ограниченим ресурсима, изабран је модел *multilingual-e5-base*. Он генерише векторе од 768 димензија, што одговара типу **Vector(768)** дефинисаном у моделу података (поглавље 5.5).

7.7 Текстуални префикси

E5 модели при претрази разликују упит од текста који се претражује употребом префикса **query** и **passage**. Код асиметричне претраге, као што је проналажење релевантног дијела документа на основу корисничког упита, префикс **query** се додаје упиту, а **passage** тексту који се претражује.

Ови префикси су важни за правилну употребу модела. Њихово изостављање не спречава генерисање вектора, али може умањити квалитет добијених репрезентација за задатак претраге. Због тога се у овом систему префикс **passage** додаје сваком *chunk*-у прије генерисања *embedding* вектора у *offline pipeline*-у (одјељак 7.10), док се префикс **query** додаје корисничком упиту прије генерисања вектора у *online pipeline*-у (одјељак 7.11).

7.8 Грануларност и *chunking*

У поглављу 6.6 је описана грануларност лексичке претраге на нивоу транскрипцијских сегмената. За потребе семантичке претраге се ти сегменти додатно групишу у веће текстуалне јединице, *chunk*-ове, које се затим претварају у *embedding* векторе.

7.8.1 Величина *chunk*-а

Величина *chunk*-а представља компромис између очувања контекста и прецизности претраге. Сувише мали *chunk*-ови могу изгубити дио контекста потребног за представљање значења текста, док сувише велики могу обухватити више различитих тема и тиме умањити прецизност векторске репрезентације [34]. Због тога се у овом систему не генерише један *embedding* за цијелу епизоду, нити за сваки кратки транскрипцијски сегмент засебно, већ се сусједни сегменти групишу у веће цјелине.

Приликом избора стратегије су разматрани приступи засновани на фиксној величини текста, преклапању сусједних *chunk*-ова и семантичком одређивању граница. Фиксно сјечење текста је једноставно, али може раздвојити повезани садржај на граници два *chunk*-а. Преклапање омогућава да се дио контекста задржи између сусједних *chunk*-ова, док семантички приступи настоје да границе одреде на основу садржаја текста, али захтијевају сложенију обраду.

У овом раду је изабран приступ који поштује постојеће границе транскрипцијских сегмената и користи преклапање између сусједних *chunk*-ова. На тај начин се избјегава произвољно прекидање сегмената када њихова дужина не премашује дозвољено ограничење, а преклапањем се задржава дио контекста са границе сусједних *chunk*-ова.

7.8.2 Алгоритам груписања сегмената

Функција `build_chunks_for_video()` из датотеке `services/chunking_service.py` редом групише транскрипцијске сегменте једне епизоде све док додавање наредног сегмента не би довело до прекорачења максималног броја токена. У имплементацији је ова граница дефинисана константом `DEFAULT_MAX_TOKENS = 500`.

Дужина *chunk*-а се мјери у токенима јер *embedding* модел обрађује текст након токенизације и има ограничену максималну дужину улаза. Модел *multilingual-e5-base* подржава улаз до 512 токена, при чему се дужи улази скраћују [40]. Због тога је у имплементацији постављена граница од 500 токена, чиме се оставља резерва у односу на максималну дужину модела за префикс *passage* и остале специјалне токене које модел додаје приликом обраде улаза.

При формирању *chunk*-а се текстови сегмената спајају редослиједом којим се појављују у транскрипту, док се почетно и крајње вријеме *chunk*-а одређују на основу првог и посљедњег сегмента који му припадају. Између два сусједна *chunk*-а се задржавају посљедња два сегмента претходног *chunk*-а, што је у имплементацији дефинисано константом `DEFAULT_OVERLAP_SEGMENTS = 2`.

Преклапање се, дакле, одређује на нивоу цијелих транскрипцијских сегмената, а не на основу фиксног броја токена. На тај начин дио контекста са краја једног *chunk*-а остаје присутан и на почетку сљедећег.

7.8.3 Гранични случајеви

Уколико један транскрипцијски сегмент сам премашује максимални број токена, он се додатно дијели на мање дијелове. Пошто за сваку појединачну ријеч није доступна засебна временска ознака, временске границе новонасталих дијелова се процјењују пропорционално унутар временског интервала оригиналног сегмента.

Након формирања *chunk*-а се број токена поново израчунава над коначним спојеним текстом, а добијена вриједност се уписује у поље `token_count`. На тај начин се чува број токена коначног текста *chunk*-а.

Уколико би сегменти пренесени из претходног *chunk*-а сами довели до прекорачења максималне величине, преклапање се за тај *chunk* изоставља. На тај начин ограничење максималног броја токена има предност над преклапањем.

7.9 Припрема базе: HNSW индекс

Табела *Chunk*, укључујући колону *embedding* типа *Vector(768)*, дефинисана је у моделу података описаном у поглављу 5.5, док су *pgvector* екстензија и конфигурација PostgreSQL сервиса представљене у поглављу 4.1. За ефикасну приближну претрагу над сачуваним *embedding* векторима је додатно креиран HNSW индекс, чији је принцип рада објашњен у одјелку 7.4.

pgvector омогућава креирање HNSW индекса директно над векторском колоном, при чему избор операторске класе зависи од мјере удаљености која се користи приликом претраге [14]. Пошто се у овом систему претрага заснива на косинусној удаљености, која је директно повезана са косинусном сличношћу описаном у одјелку 7.2, индекс је дефинисан помоћу операторске класе *vector_cosine_ops*, као што је приказано у листингу 16.

```
CREATE INDEX ix_chunks_embedding_hnsw
ON chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

Листинг 16: Креирање HNSW индекса над *embedding* векторима

За параметре *m* и *ef_construction* су задржане подразумеване вриједности *pgvector*-а, *m* = 16 и *ef_construction* = 64 [14]. Параметар *m* одређује максималан број веза са сусједним чворовима по слоју HNSW графа, док *ef_construction* одређује величину листе кандидата која се разматра током изградње индекса. Повећање вриједности ових параметара може побољшати квалитет индекса, али захтијева више меморије и продужава вријеме његове изградње [14]. У овом раду су задржане подразумеване вриједности, без додатног подешавања за конкретан корпус.

Креирање *pgvector* екстензије и HNSW индекса је укључено у *Alembic* миграцију базе. Пошто креирање екстензије захтијева специфичну PostgreSQL наредбу, она се извршава директно у миграцији, као што је приказано у листингу 17. На тај начин *Alembic* остаје једини механизам за управљање шемом базе.

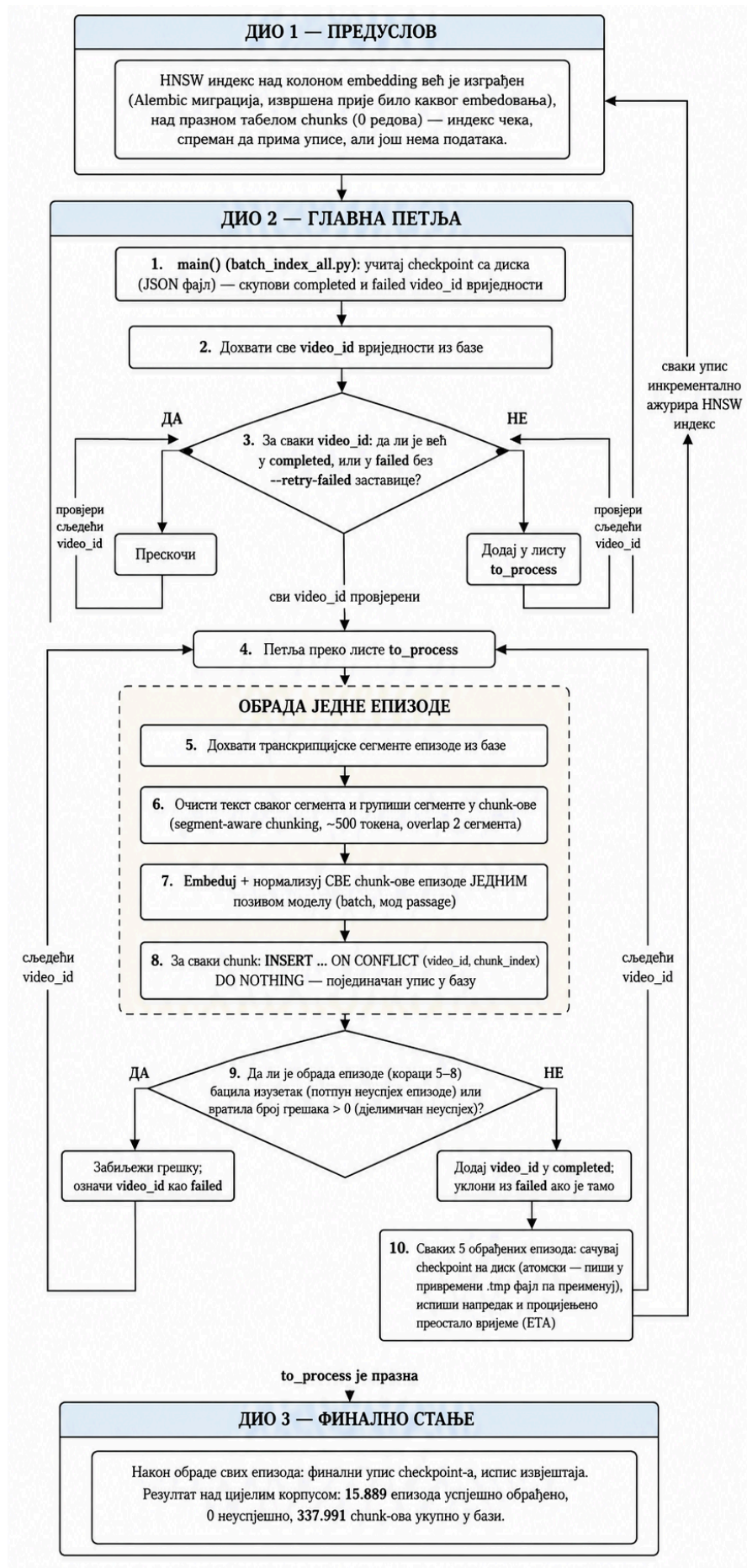
```
op.execute("CREATE EXTENSION IF NOT EXISTS vector;")
```

Листинг 17: Креирање *pgvector* екстензије у *Alembic* миграцији

7.10 Алгоритам индексирања (offline pipeline)

Генерисање *embedding* вектора за корпус се обавља у оквиру *offline pipeline*-а, независно од обраде појединачних корисничких упита. За разлику од овог поступка, *online pipeline* се покреће при сваком корисничком упиту и описан је у одјелку 7.11.

У наставку је најприје описана припрема и чишћење текста прије формирања *chunk*-ова, а затим поступак генерисања и уписа *embedding* вектора у базу за читав корпус. Комплетан ток овог процеса је приказан на слици 8.



Слика 8: Алгоритам batch embed-овања корпуса и уписа векторских репрезентација у базу

7.10.1 Чишћење текста прије *chunking*-а

Прије груписања транскрипцијских сегмената у *chunk*-ове, сваки сегмент пролази кроз функцију `clean_segment_text()`. Чишћење се обавља на нивоу појединачног сегмента како би се сачувала веза између текста и његових временских ознака, **start** и **end**.

Поступак обухвата *Unicode NFC* нормализацију, којом се различити *Unicode* записи истих карактера своде на јединствен облик, и транслитерацију, којом се текст написан ћирилицом преводи у латинични запис. Након тога се уклањају *етоји* карактери, ознаке неговорних звукова, попут „[музика]“ и „(смијех)“, сувишни симболи и размаци, као и типични артефакти аутоматске транскрипције, попут понављања истог текста и фраза „Хвала на гледању“ или „Претплатите се на канал“.

Понављања која се јављају унутар једног сегмента се уклањају током чишћења, док се понављања кроз више узастопних сегмената обрађују непосредно прије формирања *chunk*-ова. На тај начин се избегава непотребно понављање истог садржаја у тексту који се просљеђује *embedding* моделу.

7.10.2 Оркестрација поступка

Прије почетка *batch embed*-овања је *HNSW* индекс над колоном **embedding** већ креиран *Alembic* миграцијом над празном табелом **chunks**. Приликом уписа нових *embedding* вектора се индекс аутоматски ажурира.

Комплетан поступак је реализован скриптом **batch_index_all.py**. За сваку епизоду скрипта дохвата транскрипцијске сегменте из базе, чисти их и групише у *chunk*-ове. Затим се за све *chunk*-ове епизоде генеришу *embedding* вектори једним позивом функције **embed_texts()** у режиму **passage**, након чега се добијени *chunk*-ови и њихови вектори уписују у базу.

За разлику од индексирања у *Elasticsearch*-у, *batch embed*-овање се покреће као *CLI* скрипта приказана у листингу 18. Овакав приступ је изабран јер обрада цјелокупног корпуса траје приближно 20 сати, па дуготрајан *batch* процес није погодан за извршавање у оквиру синхроног *HTTP* захтјева.

```
docker compose exec backend python batch_index_all.py
```

Листинг 18: Покретање *batch embed*-овања корпуса унутар *backend* контејнера

7.10.3 Checkpoint механизам и идемпотентност

Пошто обрада цјелокупног корпуса траје више сати, скрипта користи *checkpoint* датотеку **batch_index_checkpoint.json**, која омогућава наставак рада након прекида. У њој се чувају **video_id** вриједности успјешно обрађених и неуспјешних епизода. Већ успјешно обрађене епизоде се при наредном покретању прескачу, док се обрада неуспјешних може поновити употребом заставице **--retry-failed**. Стање *checkpoint* датотеке се чува након сваких пет обрађених епизода.

Додатна заштита од дуплирања постоји на нивоу *chunk*-ова. Приликом уписа се користи *SQL* клаузула приказана у листингу 19. Комбинација **video_id** и **chunk_index** на тај начин не може довести до дуплираног уписа истог *chunk*-а. *Checkpoint* механизам омогућава наставак дуготрајне обраде, док провјера конфликта на нивоу базе обезбјеђује идемпотентност поновљеног уписа.

```
ON CONFLICT (video_id, chunk_index) DO NOTHING
```

Листинг 19: Спречавање дуплираног уписа *chunk*-ова у базу

7.10.4 Batch embed-овање и упис у базу

Embedding вектори се генеришу за све *chunk*-ове једне епизоде одједном, једним позивом модела. На тај начин се избегава засебан позив модела за сваки *chunk* и омогућава ефикаснија обрада корпуса.

Након генерисања и нормализације вектора, сваки *chunk* се појединачно уписује у базу. Приликом уписа се користи клаузула **ON CONFLICT (video_id, chunk_index) DO NOTHING**, па поновно покретање обраде не доводи до дуплирања већ постојећих *chunk*-ова.

7.10.5 Руковање грешкама и завршни извјештај

Свака епизода се обрађује независно. Епизода се означава као неуспјешна ако током њене обраде дође до изузетка или ако поступак врати једну или више грешака приликом обраде њених *chunk*-ова. У том случају се њен **video_id** додаје у скуп **failed**, а *batch* обрада се наставља са следећом епизодом.

Након успјешне обраде се **video_id** додаје у скуп **completed** и уклања из **failed** уколико се тамо налазио. Стање *checkpoint* датотеке се чува након сваких пет обрађених епизода, а по завршетку обраде се врши и финални упис *checkpoint*-а и исписује завршни извјештај.

7.10.6 Верификација

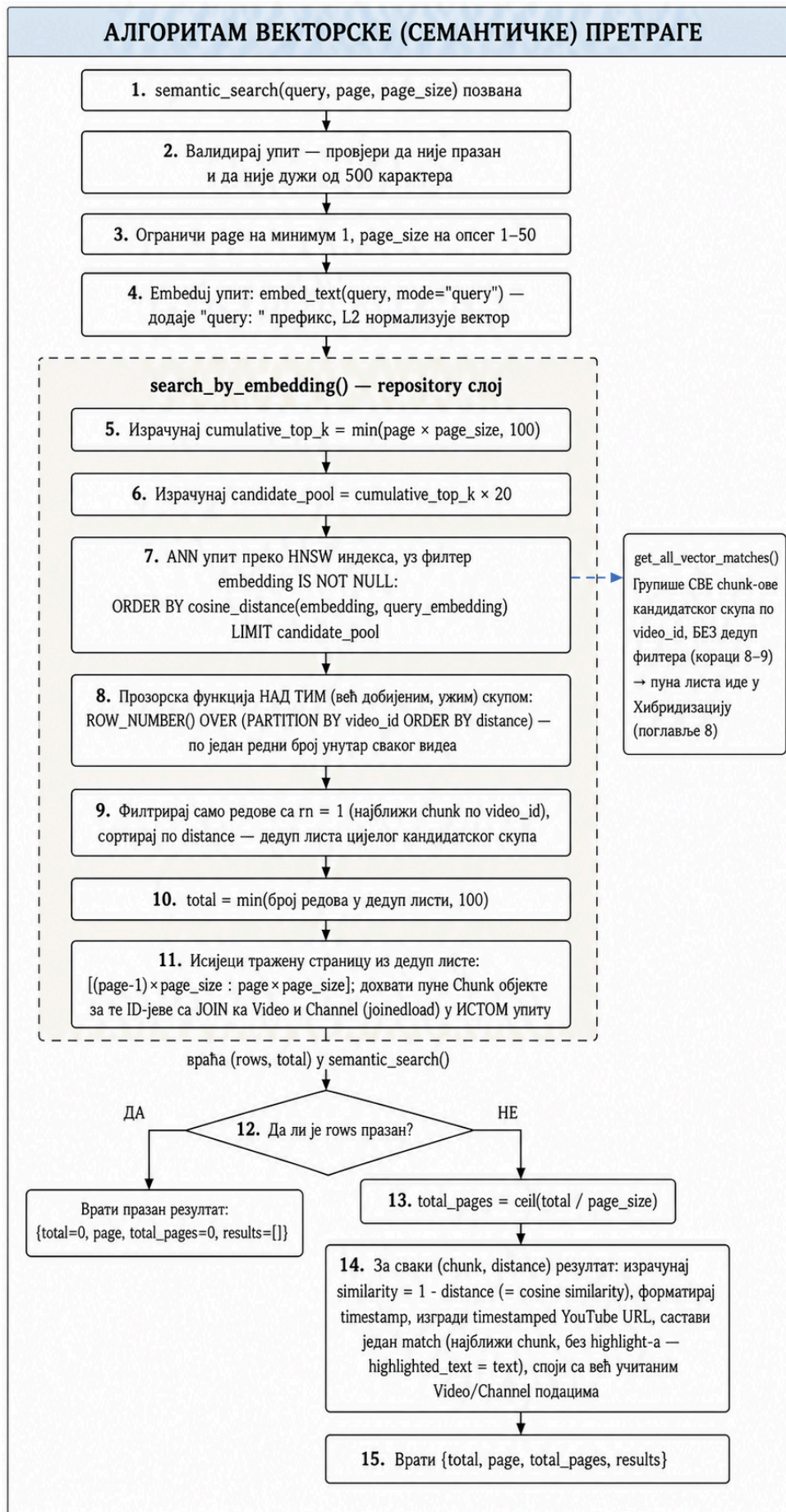
Након завршетка *batch embed*-овања је исправност поступка провјерена на основу завршног стања *checkpoint* датотеке и броја генерисаних *chunk*-ова. Резултат обраде цјелокупног корпуса је приказан у листингу 20.

```
Успјешно обрађене епизоде: 15.889
Неуспјешне епизоде:      0
Генерисани chunk-ови:    337.991
```

Листинг 20: Резултат *batch embed*-овања цјелокупног корпуса

7.11 Алгоритам претраге (*online pipeline*)

Док је претходни одјељак описао поступак генерисања и уписа *embedding* вектора корпуса, *online pipeline* се извршава при свакој корисничкој претрази. Он обухвата обраду корисничког упита, генерисање његовог *embedding* вектора, *ANN* претрагу над *HNSW* индексом и формирање листе резултата. Добијени резултати се користе и као један од улаза у хибридную претрагу, која је описана у поглављу 8. Комплетан ток алгоритма је приказан на слици 9.



Слика 9: Алгоритам векторске претраге

7.11.1 *Embedding* упита

Функција `semantic_search()` прима текстуални упит и параметре за страничење, `page` и `page_size`. Прије извршавања претраге се провјерава исправност упита и параметара страничења.

Након валидације се упит претвара у *embedding* вектор позивом функције `embed_text(query, mode="query")`. Режим *query* додаје префикс *query*: тексту упита, након чега се добијени вектор нормализује на исти начин као вектори *chunk*-ова приликом *offline embed*-овања. Добијени упитни вектор се затим користи за проналажење семантички најсличнијих дијелова транскрипта.

7.11.2 ANN претрага и груписање по епизоди

Претрага се извршава функцијом `_search_by_embedding()` над *HNSW* индексом у *PostgreSQL* бази. Њен циљ је проналажење *chunk*-ова чији су *embedding* вектори најближи вектору корисничког упита према косинусној удаљености.

Пошто једна епизода садржи више *chunk*-ова, међу најближим резултатима се може појавити више *chunk*-ова исте епизоде. Коначни резултати се, међутим, приказују на нивоу епизоде, па је потребно да се свака епизода појави само једном, са својим најрелевантнијим *chunk*-ом.

Директно груписање над цијелом табелом *chunks*, на примјер примјеном прозорске функције `ROW_NUMBER()` по `video_id`, захтијевало би израчунавање удаљености за велики број редова прије груписања. Такав приступ не би искористио основну предност *HNSW* индекса, односно проналажење ограниченог скупа приближно најближих сусједа без потпуног прегледа свих вектора.

Због тога се користи приступ *overfetch* и накнадна дедупликација. *HNSW* индексом се најприје дохвата шири кандидатски скуп најближих *chunk*-ова. Резултати се затим групишу према `video_id`, а за сваку епизоду се задржава само најближи *chunk*. Након дедупликације се из добијене листе издваја страница резултата коју је корисник затражио.

Величина кандидатског скупа се одређује помоћу константе `CANDIDATE_MULTIPLIER = 20`. Ова вриједност је изабрана на основу структуре корпуса, у којем једна епизода у просјеку садржи приближно 21 *chunk*. Дохватањем већег броја кандидата се смањује могућност да велики број најближих *chunk*-ова припада истој епизоди и да након дедупликације не остане довољно различитих епизода за попуњавање тражене странице. Већи множилац би повећао број кандидата, али и трошак претраге, па је у имплементацији изабрана вриједност 20.

Број различитих епизода које се разматрају за страничење је ограничен вриједношћу `MAX_TOTAL = 100`. За разлику од лексичке претраге, код векторске претраге не постоји природна граница која одређује да ли нека епизода одговара упиту. Сваки вектор има одређену удаљеност од упитног вектора, а резултати се рангирају од најближих ка удаљенијим. Због тога вриједност `total` у овом случају не представља укупан број свих семантички релевантних епизода у бази, већ број различитих епизода пронађених унутар ограничења постављеног вриједношћу `MAX_TOTAL`.

7.11.3 Формирање одговора

За сваки пронађени резултат се из косинусне удаљености израчунава косинусна сличност, као што је приказано у листингу 21.

```
similarity = round(1 - distance, 4)
```

Листинг 21: Израчунавање косинусне сличности на основу косинусне удаљености

Пошто вриједност `distance` представља косинусну удаљеност, одузимањем те вриједности од 1 се добија косинусна сличност. Већа вриједност сличности означава већу блискост између *embedding* вектора корисничког упита и текста *chunk*-а.

За сваку епизоду се враћа њен најрелевантнији *chunk*, заједно са текстом и временском ознаком. Вријеме почетка *chunk*-а, `start_sec`, претвара се у читљив формат помоћу функције `format_seconds()`. Директна

веза до одговарајућег тренутка у видеу се формира додавањем временског параметра **t** на *YouTube URL* епизоде.

За разлику од претраге у *Elasticsearch*-у, семантичка претрага нема директно поклапање термина на основу којег би се појединачне ријечи могле означити ознакама **<mark>**. Релевантност се одређује на основу сличности векторских репрезентација упита и цјелокупног текста *chunk*-а, а не на основу директног поклапања појединачних ријечи. Због тога поље **highlighted_text** у тренутној имплементацији садржи исти текст као поље **text**.

Метаподаци о епизоди, као што су наслов, канал и датум објављивања, налазе се у истој *PostgreSQL* бази као и *chunk*-ови. Повезани подаци о епизоди се приликом претраге дохватају помоћу **joinedload()**. За разлику од лексичке претраге, код које се резултати добијају из *Elasticsearch*-а, семантичка претрага и припадајући метаподаци се добијају из исте базе података.

7.11.4 Потпуна листа векторских погодака

Поред функције **semantic_search()**, имплементирана је и функција **get_all_vector_matches()**. Док **semantic_search()** задржава само најрелевантнији *chunk* сваке епизоде, **get_all_vector_matches()** враћа све пронађене *chunk*-ове унутар кандидатског скупа и групише их према **video_id**.

Ова функција се не користи директно за приказ резултата кориснику, већ је намијењена хибридној претрази описаној у поглављу 8, у којој је потребно располагати са више релевантних временских тренутака унутар исте епизоде.

7.12 Ограничења векторске претраге

Векторска претрага омогућава проналажење семантички сличног садржаја и када упит и транскрипт не садрже исте ријечи. Ипак, ослањање на векторску репрезентацију значења има ограничења у ситуацијама у којима је важно дословно подударање текста. Због тога се векторска и лексичка претрага могу посматрати као комплементарни приступи, чије се предности могу комбиновати у оквиру хибридне претраге.

7.12.1 Тачно подударање

Embedding модел представља читав *chunk* једним вектором фиксне димензије. Таква репрезентација омогућава поређење текстова према семантичкој сличности, али није намијењена очувању сваког детаља њиховог дословног облика.

Због тога векторска претрага није најпогоднија када је кориснику важно тачно појављивање одређене фразе, имена, броја, датума или другог специфичног термина. Семантички сличан, али текстуално различит садржај може добити висок скор, док лексичка претрага непосредно узима у обзир термине који се појављују у упиту и документу. Особина која омогућава проналажење парафраза зато може представљати недостатак када је циљ пронаћи тачан облик текста.

Лексичка претрага је посебно корисна и за специфичне термине, имена и скраћенице код којих је важно њихово непосредно појављивање у тексту. Механизам као што је **fuzziness: "AUTO"** додатно омогућава толеранцију на мање разлике у запису термина.

7.12.2 Означавање и објашњивост резултата

Elasticsearch може да означи дијелове текста у којима су пронађени термини из корисничког упита. Код векторске претраге не постоји тако директна веза између појединачних ријечи упита и резултата, јер се пореде векторске репрезентације цјелокупног упита и *chunk*-а.

Због тога поље **highlighted_text** у тренутној имплементацији садржи исти текст као **text**, без означавања конкретног дијела *chunk*-а који је допринео његовој релевантности.

Ово уједно отежава непосредно објашњавање добијених резултата. Код лексичке претраге се релевантност може повезати са терминима упита који се појављују у документу, док косинусна сличност представља

једну бројчану оцјену сличности двије векторске репрезентације. Систем зато може да рангира један *chunk* као семантички ближи упиту од другог, али из саме вриједности косинусне сличности није могуће непосредно одредити које су појединачне ријечи или дијелови текста довели до таквог резултата.

7.12.3 Рачунарски трошак

Имплементација векторске претраге захтијева додатну фазу припреме података у којој се за *chunk*-ове корпуса генеришу и складиште *embedding* вектори. У овом раду је *offline embed*-овање цјелокупног корпуса трајало приближно 20 сати. При обради корисничког упита је такође потребно генерисати његову векторску репрезентацију прије извршавања претраге над *HNSW* индексом.

Промјена *embedding* модела би захтијевала поновно генерисање вектора за цјелокупан корпус. Уколико би нови модел генерисао векторе другачије димензије, било би потребно прилагодити и тип одговарајуће колоне у бази и поново изградити векторски индекс.

Параметри *HNSW* индекса, *m* и *ef_construction*, нису систематски оптимизовани за корпус коришћен у овом раду, већ су задржане подразумијеване вриједности *pgvector*-а. Испитивање утицаја ових параметара на однос између квалитета, брзине претраге и потребних ресурса представља један од могућих праваца даљег рада.

7.12.4 Зависност од *embedding* модела и језика

Квалитет векторске претраге зависи од способности изабраног *embedding* модела да формира одговарајуће векторске репрезентације текста. Модел *multilingual-e5-base* је вишејезички модел који подржава велики број језика, укључујући српски [38].

Квалитет репрезентација може да варира у зависности од језика и карактеристика конкретног текста. У оквиру овог рада није посебно испитиван утицај регионалних варијанти, дијалеката и рјеђих језичких облика на резултате семантичке претраге, па њихова детаљнија анализа остаје могући правац будуће евалуације.

7.13 Могућа унапређења

На основу наведених ограничења могу се издвојити следећи правци даљег развоја система:

- **Оптимизација *HNSW* индекса:** параметри *m*, *ef_construction* и параметри који утичу на поступак претраге могли би се експериментално подесити за конкретан корпус и испитати њихов утицај на однос између брзине и квалитета претраге.
- **Поређење *embedding* модела:** коришћени модел *multilingual-e5-base* могао би се упоредити са већим или другим моделима, као што су *multilingual-e5-large* и *bge-m3*, уз мјерење односа између квалитета резултата и потребних рачунарских ресурса.
- **Додатно рангирање резултата (енг. *re-ranking*):** након *ANN* претраге би се мањи скуп најбољих кандидата могао додатно рангирати моделом намијењеним том задатку, чиме би се могао побољшати квалитет коначног рангирања.
- **Унапређење означавања релевантног текста:** тренутно се у векторској претрази не означава дио *chunk*-а који је најрелевантнији за упит. Додатни механизам би могао да издвоји релевантнији дио текста и тиме кориснику олакша разумијевање приказаног резултата.
- **Семантичко *chunk*-овање:** умјесто ослањања на границе транскрипцијских сегмената, при формирању *chunk*-ова би се могле узети у обзир и промјене теме или семантичке цјелине у тексту.
- **Проширивање упита:** ријетка имена, скраћенице и други специфични термини би се могли обрађивати додавањем алтернативних облика упита прије претраге.

Глава 8

Хибридизација резултата претраге (*RRF*)

У претходним поглављима су описана два приступа претрази истог корпуса: лексичка претрага заснована на алгоритму *BM25* (поглавље 6) и семантичка претрага заснована на *embedding* векторима (поглавље 7). Лексичка претрага даје предност поклапању термина из упита и документа, док семантичка претрага омогућава проналажење садржаја сличног значења и када се коришћене ријечи не поклапају. Због различитих начина представљања текста и одређивања релевантности, ова два приступа могу да се међусобно допуњују [41].

Луан и сарадници анализирају разлике између *sparse* и *dense* приступа претраживању и показују да они имају различите предности. *Sparse* репрезентације задржавају директну везу са терминима у тексту, док *dense* репрезентације омогућавају препознавање семантичке сличности и када не постоји директно поклапање термина. Њихови резултати показују и да комбиновање ова два приступа може да побољша квалитет претраге [41]. Обједињавање резултата лексичке и семантичке претраге у јединствену ранг-листу представља један од приступа хибридној претрази [42].

У систему описаном у овом раду, резултати лексичке и семантичке претраге се обједињују методом *Reciprocal Rank Fusion (RRF)*. У наставку поглавља је описан принцип рада, те разлози избора ове методе, а затим и начин њене примјене у имплементираним систему.

8.1 *Reciprocal Rank Fusion*

Reciprocal Rank Fusion (RRF) представља методу за спајање више ранжираних листа у јединствену ранг-листу [43]. За разлику од приступа који директно комбинују скорове различитих система, *RRF* се заснива на позицији резултата у свакој листи. Због тога није потребно претходно усклађивати скале *BM25* скорa и косинусне сличности [43].

За документ d и скуп ранжираних листа R , *RRF* скор се дефинише као [43]:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_{r(d)}}$$

гдје $\text{rank}_{r(d)}$ представља позицију документа d у ранг-листи r , док је k константа којом се ублажава утицај разлика између највиших позиција. Ако се документ не појављује у одређеној листи, та листа не доприноси његовом *RRF* скору.

8.1.1 Улога константе k

Константа k ублажава разлику између доприноса резултата који се налазе на највишим позицијама ранг-листе. Без ове константе би прва позиција доприносила вриједношћу 1, а друга $\frac{1}{2}$. Са $k = 60$, њихови доприноси износе $\frac{1}{61}$ и $\frac{1}{62}$, па разлика између сусједних позиција на врху листе има мањи утицај на коначни скор.

Сормаск и сарадници су у оригиналном раду за *RRF* користили вриједност $k = 60$. Она је изабрана на основу пилот-експеримента, а аутори наводе да се показала близу оптималне и да резултати нису били нарочито осјетљиви на избор ове константе [43]. Иста вриједност је коришћена и у систему описаном у овом раду.

8.1.2 Интуиција кроз примјер

Претпоставимо да лексичка претрага за исти упит рангира епизоде A , B и C на позиције 1, 2 и 3, док их семантичка претрага поставља на позиције 2, 4 и 1. За $k = 60$ се добија:

$$RRF(A) = \frac{1}{60 + 1} + \frac{1}{60 + 2} = 0.03252$$

$$RRF(B) = \frac{1}{60 + 2} + \frac{1}{60 + 4} = 0.03175$$

$$RRF(C) = \frac{1}{60 + 3} + \frac{1}{60 + 1} = 0.03227$$

Конечан поредак је, према томе, $A > C > B$. Епизоде A и C имају највећи скор јер су високо позициониране у обје листе. RRF на тај начин омогућава да резултат који је добро рангиран у више система добије допринос од сваког од њих, без директног поређења њихових оригиналних скорова.

8.2 Избор RRF -а у односу на алтернативне приступе

При избору начина спајања резултата су разматрана два алтернативна приступа: комбиновање оригиналних скорова лексичке и семантичке претраге и додатно рангирање резултата помоћу *cross-encoder* модела.

8.2.1 Комбиновање скорова

Један од начина спајања резултата јесте комбиновање $BM25$ скорa и косинусне сличности, на примјер њиховом пондерисаном сумом. Међутим, ове вриједности нису директно упоредиве јер потичу из различитих модела рангирања и могу имати различите расподеле и опсеге. Због тога је при оваквом приступу потребно претходно нормализовати скорове, на примјер *min-max* нормализацијом, а затим одредити тежину сваког сигнала [44].

RRF избјегава потребу за усклађивањем скорова јер користи позицију резултата у појединачним ранг-листама, а не њихове оригиналне вриједности. Због једноставности и одсуства потребе за подешавањем тежина, овај приступ је погодан за систем развијен у оквиру овог рада.

8.2.2 *Cross-encoder reranking*

Друга разматрана могућност је била примјена *cross-encoder reranking*. Код овог приступа се прво постојећим механизмом претраге издваја мањи број кандидата, након чега се они додатно рангирају помоћу *cross-encoder* модела.

Разлика у односу на *bi-encoder* приступ коришћен у семантичкој претрази овог система јесте у начину обраде упита и текста. Код *bi-encoder* приступа се упит и текст кодирају независно у засебне *embedding* векторе, након чега се њихова сличност израчунава поређењем добијених вектора. *Cross-encoder*, на супрот томе, прима упит и текст кандидата заједно и директно процјењује колико је тај текст релевантан за конкретан упит. Оваква директна интеракција може да омогући прецизнију процјену релевантности, али захтијева засебну обраду сваког пара упита и кандидата, због чега је рачунски захтјевнија.

Истраживања показују да *cross-encoder* модели могу да постигну добре резултате и у *zero-shot* условима, односно када се примјењују на податке из домена за који нису посебно обучавани или прилагођавани [45]. *BEIR* евалуација такође показује да модели засновани на поновном рангирању у просјеку постижу добре резултате у таквим условима, али уз веће рачунске трошкове [46].

Увођење *cross-encoder* модела у овај систем би захтијевало додатни корак обраде над кандидатима добијеним у првој фази претраге, као и избор и евалуацију модела погодног за српски језик и корпус коришћен у овом раду. Због тога је у тренутној имплементацији предност дата једноставнијем приступу заснованом на RRF -у, док *cross-encoder reranking* представља могућност за будуће унапређење система.

8.2.3 Ограничења RRF -а

Иако је једноставан за примјену, RRF није нужно оптималан начин спајања резултата у сваком систему. Bruch и сарадници показују да његови резултати могу да зависе од избора параметара, као и да комбинација

нормализованих лексичких и семантичких скорова може да надмаши *RRF* када су доступни подаци за подешавање параметара фузије [47].

У оквиру овог рада је *RRF* ипак изабран јер не захтијева нормализацију различитих скорова нити податке за обучавање параметара фузије, а његова примјена над постојећим ранг-листама је једноставна и рачунски јефтина. Оптимизација начина спајања резултата на основу означеног скупа упита и релевантних епизода остаје могући правац даљег развоја система.

8.3 Архитектура: три ендпоинта

Систем излаже три одвојена ендпоинта за претрагу: ***/search*** за лексичку претрагу описану у поглављу 6, ***/search/semantic*** за семантичку претрагу описану у поглављу 7 и ***/search/hybrid*** за хибридную претрагу описану у овом поглављу. Оваква организација омогућава независно коришћење и тестирање сваког приступа, као и њихово директно поређење приликом евалуације система у поглављу 9.

Хибридна претрага је имплементирана у сервису ***hybrid_search_service.py***, који користи постојеће функције ***search_podcasts()*** и ***semantic_search()***. На тај начин је избјегнуто дуплирање логике лексичке и семантичке претраге, а њихови резултати се у хибридном сервису само обједињују и поново рангирају примјеном *RRF* методе.

Сва три ендпоинта користе исти формат одговора (***SearchResult*** и ***SearchResponse***), са пољима као што су ***videoId***, ***channelName***, ***episodeTitle***, ***relevanceScore*** и ***matches***. Захваљујући јединственом формату, *frontend* апликација и скрипта за евалуацију могу на исти начин да обрађују резултате сва три приступа претрази.

8.4 Ниво фузије резултата

Лексичка и семантичка претрага се не извршавају над истом јединицом текста. Лексичка претрага ради над појединачним сегментима транскрипта, док семантичка претрага ради над *chunk*-овима насталим груписањем више узастопних сегмената. Разлози за избор ових грануларности су описани у поглављима 6 и 7.

Због различите грануларности, резултате није практично директно спајати на нивоу сегмента или *chunk*-а. Један *chunk* може да обухвата више сегмената, па резултати два система немају директно једнозначно пресликавање.

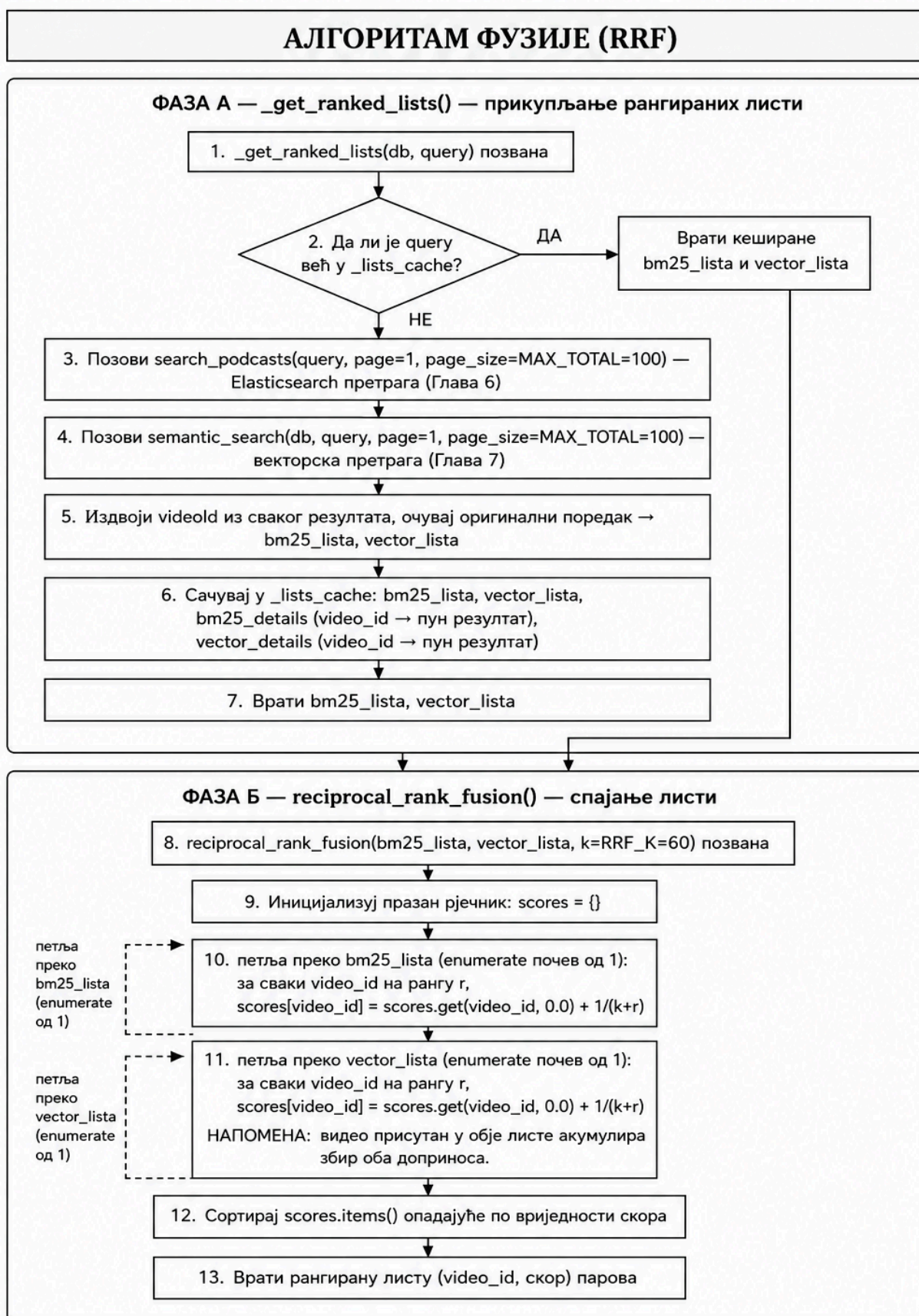
8.4.1 Фузија на нивоу епизоде

Резултати лексичке и семантичке претраге се спајају на нивоу епизоде, која представља њихов заједнички ниво. Сваки сегмент и сваки *chunk* припадају тачно једној епизоди и садрже њен ***video_id***, што омогућава да се резултати прије примјене *RRF* методе групишу и рангирају према епизоди.

Овај приступ одговара и начину приказа резултата у корисничком интерфејсу, гдје основни резултат представља епизода, док пронађени сегменти и *chunk*-ови указују на релевантне дијелове њеног транскрипта.

8.5 Алгоритам фузије

Имплементација *RRF* методе је подијељена у два корака. Најприје се добијају рангиране листе епизода из лексичке и семантичке претраге, након чега се оне спајају у јединствену листу примјеном *RRF* формуле. Алгоритам је приказан на слици 10.



Напомена: Кеш приказан на дијаграму чува искључиво резултате претраге, независно од филтера. Примјена филтера над већ фузионисаном листом и засебан кеш за филтриране резултате описани су у одјелку 8.9.

Слика 10: Алгоритам фузије резултата примјеном *RRF* методе

8.5.1 Прикупљање ранжираних листа

Функција `get_ranked_lists()` позива `search_podcasts()` и `semantic_search()` са `page=1` и `page_size=MAX_TOTAL`, при чему је `MAX_TOTAL = 100`. На тај начин се за фузију прикупља до 100 најбоље ранжираних епизода из сваког система претраге, као што је приказано у листингу 22.

```
def _get_ranked_lists(db: Session, query: str) -> tuple[list[str], list[str]]:
    cached = _lists_cache.get(query)
    if cached is not None:
        return cached["bm25_lista"], cached["vector_lista"]

    bm25_response = search_podcasts(query=query, page=1, page_size=MAX_TOTAL)
    vector_response = semantic_search(db, query=query, page=1, page_size=MAX_TOTAL)

    bm25_lista = [r["videoId"] for r in bm25_response["results"]]
    vector_lista = [r["videoId"] for r in vector_response["results"]]

    _lists_cache[query] = {
        "bm25_lista": bm25_lista,
        "vector_lista": vector_lista,
        "bm25_details": {r["videoId"]: r for r in bm25_response["results"]},
        "vector_details": {r["videoId"]: r for r in vector_response["results"]},
    }
    return bm25_lista, vector_lista
```

Листинг 22: Прикупљање и кеширање ранжираних листа *BM25* и векторске претраге

Из добијених резултата се издвајају **videoId** вриједности уз очување њиховог оригиналног поретка. Тако настају **bm25_lista** и **vector_lista**, које представљају улаз у *RRF* алгоритам. Остали подаци о резултатима се чувају у **bm25_details** и **vector_details** како би се касније могли искористити за формирање коначног одговора. Механизам кеширања ових података је описан у одјељку 8.8.

8.5.2 Спајање листа

Спајање двије ранг-листе је реализовано функцијом `reciprocal_rank_fusion()`, приказаном у листингу 23:

```
def reciprocal_rank_fusion(
    bm25_lista: list[str], vector_lista: list[str], k: int = RRF_K
) -> list[tuple[str, float]]:
    scores: dict[str, float] = {}

    for rank, video_id in enumerate(bm25_lista, start=1):
        scores[video_id] = scores.get(video_id, 0.0) + 1.0 / (k + rank)

    for rank, video_id in enumerate(vector_lista, start=1):
        scores[video_id] = scores.get(video_id, 0.0) + 1.0 / (k + rank)

    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

Листинг 23: Имплементација *Reciprocal Rank Fusion (RRF)* алгоритма за спајање ранжираних листа

Функција пролази кроз обје листе и за сваку епизоду израчунава допринос $\frac{1}{k + \text{rank}}$. Ако се иста епизода појављује у обје листе, њени доприноси се сабирају, док епизода присутна само у једној листи добија допринос искључиво на основу своје позиције у тој листи. Након обраде свих резултата, епизоде се сортирају опадајуће према добијеном *RRF* скору, чиме се формира јединствена ранг-листа хибридне претраге.

8.6 Пагинација хибридних резултата

Пагинација хибридне претраге се примјењује тек након спајања резултата. Просљеђивање параметара **page** и **page_size** директно лексичкој и семантичкој претрази није погодно, јер се коначни поредак епизода

одређује тек након примјене *RRF* методе. Епизода која има нижу позицију у једној листи може након фузије бити високо рангирана захваљујући својој позицији у другој листи.

Због тога се за сваки нови упит најприје прикупља до **MAX_TOTAL** = **100** најбоље ранжираних епизода из сваког система, као што је приказано у листингу 24:

```
bm25_response = search_podcasts(
    query=query,
    page=1,
    page_size=MAX_TOTAL
)

vector_response = semantic_search(
    db,
    query=query,
    page=1,
    page_size=MAX_TOTAL
)
```

Листинг 24: Покретање лексичке и векторске претраге за исти кориснички упит

Над тако добијеним листама се извршава *RRF* фузија, чиме настаје јединствена ранг-листа. Пагинација се затим примјењује над већ спојеним резултатима, као што је приказано у листингу 25.

```
total = len(rrf_rezultat)
offset = (page - 1) * page_size
stranica = rrf_rezultat[offset : offset + page_size]
```

Листинг 25: Пагинација резултата након *RRF* фузије

На овај начин се све странице истог упита добијају из истог кандидатског скупа, па се поредак резултата и вриједност **total** не мијењају приликом преласка са једне странице на другу. При томе **total** представља број јединствених епизода добијених фузијом прикупљених резултата, а не укупан број потенцијално релевантних епизода у читавом корпусу.

Овај приступ захтијева да се већ при првом позиву прикупи до 100 резултата из сваког система, чак и ако корисник прегледа само прву страницу. Тај компромис је прихваћен како би се обезбиједили стабилан поредак и досљедна пагинација, док се поновљени позиви за исти упит оптимизују механизмом кеширања описаним у наредном одјељку.

8.7 Кеширање

Како би се избјегло поновно извршавање лексичке и семантичке претраге приликом преласка између страница истог упита, прикупљене ранг-листе се привремено чувају у меморији. Кеш је имплементиран као *Python* рјечник.

Кључ представља текст корисничког упита, док се за њега чувају лексичка и семантичка ранг-листа (**bm25_lista** и **vector_lista**), као и подаци о резултатима потребни за касније формирање одговора. При поновном позиву са истим упитом се ови подаци преузимају из меморије, без поновног извршавања претраге у *Elasticsearch*-у и *PostgreSQL*-у.

У овој имплементацији је коришћен једноставан меморијски кеш умјесто посебног система као што је *Redis*. Овакво рјешење је довољно за локално покретање и евалуацију система, а истовремено не уводи додатну инфраструктурну зависност.

Кеширају се изворне ранг-листе, а не коначна листа добијена *RRF* методом. На тај начин се *RRF* скор може поново израчунати над већ доступним резултатима, без новог извршавања лексичке и семантичке претраге.

Овако реализован кеш нема ограничење величине нити механизам истицања записа, а везан је за меморију појединачног *backend* процеса. Због тога представља рјешење прилагођено тренутном начину употребе

система. За вишекорисничко или продукцијско окружење кеширање би се могло проширити ограничењем величине, политиком избацивања старих записа или употребом посебног система за кеширање.

8.8 Филтрирање након фузије

Поред текстуалног упита, систем подржава филтрирање резултата према називу епизоде, каналу и датуму објаве. У хибридној претрази се филтери примјењују након *RRF* фузије, односно над већ формираном ранг-листом епизоде.

Лексичка и семантичка претрага се зато најприје извршавају без ових филтера, након чега се њихове ранг-листе спајају *RRF* методом. Филтери затим само уклањају епизоде које не задовољавају задате услове, без промјене међусобног поретка преосталих резултата.

Овај приступ омогућава да основни *RRF* ранг за исти текстуални упит остане непромијењен без обзира на накнадно изабране филтере. Да су филтери примјењивани прије претраге и фузије, свака комбинација филтера би формирала другачији кандидатски скуп, па би се мијењале и позиције епизоде које улазе у *RRF* формулу.

Провјера филтера је реализована функцијом `_passes_filters()`, као што је приказано у листингу 26.

```
def _passes_filters(details, episode_name, channel_name, date_from, date_to) -> bool:
    if episode_name and episode_name.strip().lower() not in details["episodeTitle"].lower():
        return False
    if channel_name and details["channelName"] != channel_name:
        return False
    if date_from or date_to:
        published_str = details.get("publishedAt")
        if not published_str:
            return False
        published_date = datetime.strptime(published_str, "%Y-%m-%d").date()
        if date_from and published_date < datetime.strptime(date_from, "%Y-%m-%d").date():
            return False
        if date_to and published_date > datetime.strptime(date_to, "%Y-%m-%d").date():
            return False
    return True
```

Листинг 26: Провјера филтера над резултатима хибридне претраге

Филтер по називу епизоде се заснива на провјери да ли се задати текст налази у називу, без употребе анализатора лексичке претраге. Филтер по каналу захтијева тачно поклапање назива канала, док се датум објаве провјерава у односу на задату доњу и горњу границу. Ако датумски филтер постоји, а епизода нема познат датум објаве, она се искључује из резултата.

8.8.1 Кеширање филтрираних резултата

Исти текстуални упит може да се користи са различитим комбинацијама филтера. Због тога се филтрирани резултати чувају одвојено од нефилтрираних ранг-листа, користећи кључ који обухвата упит и све параметре филтрирања.

Промјена било ког филтера формира нови кључ. Филтрирање се тада поново примјењује над већ кешираним резултатима претраге, без поновног позивања лексичког и семантичког система.

8.9 Претрага без текстуалног упита

Напредна претрага омогућава задавање само филтера, без текста упита. У том случају не постоји текстуални сигнал на основу којег би лексичка или семантичка претрага одређивале релевантност, па се *Elasticsearch* и векторска претрага не извршавају. Умјесто тога, систем директно претражује табелу **Video** у *PostgreSQL* бази и примјењује задате филтере према називу епизоде, каналу и датуму објаве. Ако нису задати ни текстуални упит ни филтери, враћа се празна листа резултата.

Пошто у овом случају не постоји скор релевантности, пронађене епизоде се сортирају према датуму објаве, од најновијих ка старијим. Формат одговора остаје исти као код осталих начина претраге, чиме се задржава јединствен начин обраде резултата на страни *frontend* апликације.

При филтрирању према горњој граници датума се обухвата читав дан наведен у параметру **date_to**. Пошто се у бази чува и вријеме објаве, као искључива горња граница се користи почетак наредног дана. На тај начин епизоде објављене током последњег дана задатог периода неће бити изостављене због времена објаве.

8.10 Приказ релевантних исјечака по епизоди

Хибридни резултат, поред ранга епизоде, садржи и временске исјечке (**matches**) који указују на дијелове транскрипта препознате као релевантне. Пошто лексичка и семантичка претрага ове исјечке проналазе независно, у хибридном одговору се они обједињују и приказују у јединственој листи.

Са лексичке стране се користе сегменти добијени кроз *Elasticsearch inner_hits*. Како би хибридни резултат могао да прикаже већи број релевантних дијелова исте епизоде, параметар **size** је повећан са 3 на 100. На тај начин може да се врати до 100 најбоље ранжираних сегмената за једну епизоду.

Са семантичке стране се користи функција **get_all_vector_matches()**, која, за разлику од поступка рангирања епизоде, не задржава само најбољи *chunk* по епизоди, већ групише све пронађене *chunk*-ове из кандидатског скупа према **video_id**.

8.10.1 Усклађивање векторског кандидатског скупа

Кандидатски скуп који користи **get_all_vector_matches()** је усклађен са скупом који се користи приликом семантичког рангирања. Његова величина се зато не задаје независно, већ се изводи из исте константе, као што је приказано у листингу 27.

```
from app.repositories.chunk_repository import CANDIDATE_MULTIPLIER
```

```
VECTOR_CANDIDATE_POOL = MAX_TOTAL * CANDIDATE_MULTIPLIER
```

Листинг 27: Одређивање величине скупа кандидата за векторску претрагу

На овај начин се промјена **CANDIDATE_MULTIPLIER** примјењује на оба поступка, чиме се избјегава неслагање између скупа *chunk*-ова коришћених за рангирање и скупа из којег се формирају исјечци за приказ.

8.10.2 Спајање и сортирање исјечака

За сваку епизоду се лексички и семантички исјечци спајају у једну листу и сортирају према временској позицији унутар видеа, као што је приказано у листингу 28.

```
bm25_matches = bm25_details.get(video_id, {}).get("matches", [])
vector_matches = vector_matches_by_video.get(video_id, [])
```

```
combined = bm25_matches + vector_matches
combined.sort(key=lambda m: m["timestamp_seconds"])
```

```
result["matches"] = combined
```

Листинг 28: Спајање и хронолошко сортирање поклапања лексичке и векторске претраге

Корисник тако добија хронолошки уређен преглед релевантних дијелова епизоде, без обзира на то да ли су пронађени лексичком или семантичком претрагом.

Прикупљање додатних векторских исјечака захтијева још један упит над табелом **Chunk**. Добијени подаци се кеширају заједно са осталим резултатима упита, па се овај корак не понавља приликом преласка између страница исте претраге.

8.11 Ограничења и правци даљег развоја

Одређена ограничења семантичке претраге описана у поглављу 7 се преносе и на хибридни приступ. Прије свега, тренутна имплементација не користи минимални праг сличности за семантичке резултате, па и слабије повезани кандидати могу да учествују у *RRF* фузији. Одређивање одговарајућег прага на основу евалуационог скупа представља један од могућих праваца унапређења.

Вриједност параметра $k = 60$ је преузета из оригиналног рада о *RRF* методи и није посебно оптимизирана за корпус коришћен у овом раду [43]. Даља евалуација би могла да обухвати испитивање различитих вриједности овог параметра, као и других начина фузије. Посебно, комбинација нормализованих лексичких и семантичких скорова може да надмаши *RRF* када су доступни подаци за подешавање параметара фузије [47]. Додатну могућност представља *cross-encoder reranking*, описан у одјелку 8.2.2.

Простор за унапређење постоји и у погледу скалабилности. Меморијски кеш нема ограничење величине нити механизам истицања записа, па би у вишекорисничком или продукцијском окружењу било потребно увести одговарајућу политику управљања кешом или посебан систем за кеширање. Такође, код већег корпуса би требало испитати однос између величине кандидатског скупа, времена одзива и квалитета хибридне претраге.

У претходним поглављима су описана три начина претраге реализована у оквиру система: лексичка претрага заснована на алгоритму *BM25*, семантичка претрага заснована на векторским репрезентацијама текста и хибридна претрага која њихове резултате обједињује примјеном методе *Reciprocal Rank Fusion* (*RRF*). Како би се испитало у којој мјери различити приступи проналазе и правилно рангирају релевантне епизоде, спроведена је евалуација над ручно означеним скупом упита.

Циљ евалуације није био само да се утврди који приступ проналази већи број релевантних епизода, већ и да се испита колико високо се релевантни резултати појављују у ранг-листи. Због тога су коришћене три мјере које ће бити описане у наставку рада:

- ***Recall@10***;
- ***MRR@10***;
- ***nDCG@10***.

На овај начин је могуће посматрати различите аспекте квалитета претраге и директно упоредити лексички, семантички и хибридни приступ.

9.1 Евалуациони скуп

За потребе евалуације је формиран референтни скуп од 20 корисничких упита. Евалуација је ограничена на епизоде канала „Лукавство ума“, што је омогућило ручни преглед резултата и формирање референтног скупа релевантних епизода за сваки упит.

Упити су одабрани тако да обухвате различите начине на које корисник може да формулише претрагу. Скуп садржи краће термилошке упите, као што су „вјештачка интелигенција“, „директна демократија“, „култура поништавања“, „национални мит“, „владавина права“ и „стоицизам“, али и дуже упите формулисане природним језиком, као што су „како се формира лични идентитет“, „може ли већина угрозити демократију“ и „зашто је савременом човјеку тешко да вјерује друштвеном систему“. На тај начин евалуациони скуп обухвата и упите код којих се очекује директно подударање термина и упите код којих је за проналажење резултата потребно препознати семантичку повезаност текста.

За сваки упит су ручно одређене релевантне епизоде и додјељена им је једна од двије оцјене релевантности:

- оцјена **2** би стајала уз епизоде које су непосредно и изразито релевантна за упит;
- оцјена **1** би стајала уз епизоде које су тематски повезане са упитом, али је веза слабија или се тражена тема појављује као дио ширег контекста.

Епизоде које нису означене се сматрају нерелевантним за потребе евалуације. Овако формиран скуп представља референтни скуп на основу којег се резултати сва три приступа пореде под једнаким условима.

За сваки од 20 упита евалуациона скрипта позива лексички, семантички и хибридни *API endpoint* и преузима првих десет резултата. Сви позиви користе исти филтер канала и исти број резултата, тако да разлике у добијеним мјерама произилазе из начина претраге и рангирања, а не из различитих услова извршавања.

9.2 Мјере евалуације

За поређење система су коришћене мјере ***Recall@10***, ***MRR@10*** и ***nDCG@10***. За све три мјере је коришћена гранична позиција $k = 10$, односно евалуација је спроведена над првих десет резултата које систем враћа.

9.2.1 Recall@10

Recall@10 мјери колики дио релевантних епизода се појављује међу првих десет резултата које систем врати [48].

$$\text{Recall@10} = \frac{\text{број релевантних епизода у првих 10 резултата}}{\text{укупан број релевантних епизода}}$$

При израчунавању ове мјере се релевантним сматрају епизоде са оцјеном **1** или **2**. Виша вриједност означава да систем успијева да пронађе већи дио релевантног садржаја.

9.2.2 MRR@10

Мјера *Mean Reciprocal Rank (MRR)* се заснива на позицији првог релевантног резултата у ранг-листи [49]. За појединачни упит се користи реципрочна вриједност његовог ранга. Ако се прва релевантна епизода налази на првој позицији, вриједност је **1**, ако је на другој, вриједност је **1/2**, ако је на трећој, **1/3**, итд. Уколико међу првих десет резултата нема релевантне епизоде, вриједност је **0**.

Коначни **MRR@10** представља просјек ових вриједности за све евалуационе упите. Ова мјера је посебно значајна за претраживачки систем, јер показује колико брзо корисник долази до првог корисног резултата.

9.2.3 nDCG@10

Мјера *Normalized Discounted Cumulative Gain (nDCG)* је намијењена евалуацији ранжираних резултата са различитим степенима релевантности [50]. У овој евалуацији се користе оцјене **2** и **1**, па је пожељно да епизоде са оцјеном **2** буду ранжиране испред оних са оцјеном **1**.

Допринос релевантне епизоде се постепено смањује што се она налази ниже у ранг-листи. Добијени *DCG* се затим дијели са идеалним *DCG*-ом (*IDCG*), односно резултатом који би био добијен када би све релевантне епизоде биле поређане оптималним редослиједом. На тај начин се добија нормализована вриједност између **0** и **1**, при чему вриједност ближа јединици означава квалитетније рангирање.

У имплементацији је за **nDCG@10** коришћен линеарни *gain*, односно директна оцјена релевантности **1** или **2**. Овај избор задржава једноставну интерпретацију двостепене скале коришћене приликом ручног означавања епизода.

9.3 Резултати

Просјечне вриједности мјера над свих 20 упита су приказане у табели 2.

Табела 2: Просјечни резултати евалуације лексичке, семантичке и хибридне претраге

Систем	Recall@10	MRR@10	nDCG@10
BM25	0,3257	0,6667	0,3889
Семантичка	0,5698	0,9500	0,6318
Хибридна	0,5682	1,0000	0,6453

Резултати показују разлике између три приступа. Лексичка претрага заснована на алгоритму *BM25* је остварила најниже просјечне вриједности све три мјере. Просјечни **Recall@10** од 0,3257 показује да се међу првих десет резултата у просјеку проналази приближно трећина епизода које су у референтном скупу означене као релевантне. Вриједност **MRR@10** од 0,6667 показује да се први релевантан резултат често налази релативно високо у ранг-листи, али не досљедно на самом врху. Просјечни **nDCG@10** од 0,3889 указује да је и укупан поредак релевантних резултата слабији него код друга два приступа.

Семантичка претрага је остварила боље просјечне резултате од лексичке претраге према све три мјере. Просјечни **Recall@10** износи 0,5698, **MRR@10** 0,9500, а **nDCG@10** 0,6318. Висока вриједност **MRR@10** показује да се код већине испитиваних упита релевантна епизода појављује већ на самом врху ранг-листе. Исто-

времено, већи **Recall@10** у односу на **BM25** показује да семантичка претрага проналази већи дио ручно означених релевантних епизода.

Хибридна претрага је остварила највеће просјечне вриједности **MRR@10** и **nDCG@10**. Вриједност **MRR@10** износи 1,0000, што значи да се за свих 20 тестираних упита бар једна релевантна епизода налазила на првој позицији хибридне ранг-листе. Просјечни **nDCG@10** износи 0,6453, што је највећа вриједност међу три испитивана приступа. Са друге стране, **Recall@10** износи 0,5682 и незнатно је нижи од вриједности 0,5698 коју је остварила самостална семантичка претрага.

У односу на **BM25**, хибридни приступ је остварио веће просјечне вриједности све три мјере. У поређењу са семантичком претрагом, разлика је мања и зависи од посматране мјере. Семантичка претрага има незнатно већи **Recall@10**, док хибридна претрага има већи **MRR@10** и **nDCG@10**. То показује да **RRF** фузија у оквиру испитиваног скупа није повећала просјечан удио пронађених релевантних епизода у односу на самосталну семантичку претрагу, али је побољшала њихово рангирање, посебно позицију првог релевантног резултата.

9.4 Анализа појединачних упита

Анализа појединачних упита показује да предност једног приступа зависи и од начина на који је упит формулисан. Код упита „директна демократија“ сва три приступа постављају епизоду „56 О директној демократији“ на прво мјесто. **BM25** остварује **Recall@10** од 0,70 и **nDCG@10** од 0,7210, семантичка претрага **Recall@10** од 0,80 и **nDCG@10** од 0,8074, док хибридна претрага задржава **Recall@10** од 0,80 и повећава **nDCG@10** на 0,9000. У овом случају сва три приступа правилно препознају најочигледније релевантну епизоду, док семантичка и хибридна претрага проналазе већи дио релевантних епизода, а хибридна претрага даје најквалитетнији њихов поредак.

Код упита „национални мит“ разлика је израженија. **BM25** поставља епизоду „7 О националном миту“ тек на трећу позицију, због чега је **MRR@10** једнак 0,3333. Семантичка претрага остварује **Recall@10** од 0,8750, **MRR@10** од 1,0000 и **nDCG@10** од 0,9482. Хибридна претрага има исти **Recall@10** од 0,8750 и **MRR@10** од 1,0000, али нешто нижи **nDCG@10** од 0,9314. Овај примјер показује да семантички приступ може значајно побољшати препознавање тематски релевантних епизода, док фузија не мора у сваком појединачном случају додатно побољшати поредак који је већ формирала семантичка претрага.

Разлика између приступа је посебно видљива код дужих упита формулисаних природним језиком. За упит „може ли већина угрозити демократију“ **BM25** не враћа ниједан резултат међу првих десет, па су све три мјере једнаке нули. Семантичка и хибридна претрага, међутим, остварују **Recall@10** од 0,60 и **MRR@10** од 1,00, при чему се епизода „56 О директној демократији“ налази на првој позицији. Овај примјер показује предност семантичког приступа код описно формулисаних упита код којих релевантан садржај не мора да садржи исту формулацију као кориснички упит.

Слично понашање се уочава код упита „какву улогу интелектуалци имају у савременом друштву“. **BM25** не враћа ниједан резултат међу првих десет, док семантичка и хибридна претрага на прво мјесто постављају епизоду „44 О улози интелектуалца у друштву“. Оба приступа у овом случају остварују **Recall@10** од 0,20, **MRR@10** од 1,00 и **nDCG@10** од 0,3707. Иако је пронађен само дио свих ручно означених релевантних епизода, непосредно релевантна епизода је постављена на сам врх ранг-листе.

Још један примјер различитог понашања приступа је упит „утицај друштвених мрежа на ментално здравље“. **BM25** остварује **Recall@10** од 0,20, семантичка претрага 0,70, а хибридна 0,60. Семантичка претрага, дакле, проналази највећи број релевантних епизода, али јој је **MRR@10** 0,50, јер се први релевантан резултат налази на другој позицији. Хибридна претрага има нешто нижи **Recall@10** од 0,60, али **MRR@10** од 1,00 и **nDCG@10** од 0,6417. Овај примјер добро илуструје разлику између мјера: већи број пронађених релевантних резултата не мора истовремено значити и боље позиционирање најрелевантнијих резултата.

9.5 Ограничења евалуације

Приликом тумачења резултата потребно је узети у обзир ограничења спроведене евалуације. Референтни скуп обухвата 20 упита и формиран је над епизодама једног канала. Такав приступ је омогућио ручни преглед садржаја и контролисано означавање релевантности, али добијене вриједности не треба посматрати као општу процјену квалитета претраге над свим каналима у корпусу.

Додатно, оцјене релевантности су одређене ручно, па сама процјена релевантности садржи одређен степен субјективности. Нарочито код сложенијих тематских упита граница између дјелимично релевантне и нерелевантне епизоде није увијек потпуно оштра. У будућој евалуацији би се овај утицај могао смањити укључивањем више оцјењивача и мјерењем њихове међусобне сагласности.

Такође, евалуација је ограничена на првих десет резултата. Овај избор одговара практичној употреби претраживача, у којој су кориснику најзначајнији резултати на врху листе, али не показује колико релевантних епизода систем може да пронађе у дубљим дијеловима ранг-листе.

Упркос наведеним ограничењима, исти референтни скуп, исти филтер канала и исте мјере су примијењени на сва три приступа. Због тога резултати омогућавају директно поређење њиховог понашања у контролисаним условима. У оквиру испитиваног скупа семантичка претрага је остварила незнатно већи просјечни **Recall@10**, док је хибридна претрага остварила највеће просјечне вриједности **MRR@10** и **nDCG@10**. Ови резултати показују да лексички и семантички приступ пружају комплементарне сигнале релевантности, али и да њихова фузија не мора надмашити најбољи појединачни приступ према свакој мјери.

У овом раду пројектован је и имплементиран систем за текстуалну претрагу транскрипата *YouTube* подкаста на српском језику. Основни циљ је био омогућити кориснику да на основу текстуалног упита пронађе релевантне епизоде и одговарајуће временске исјечке унутар њих, без потребе за ручним прегледањем вишесатног аудио-видео садржаја.

Реализација система је обухватила цјелокупан ток обраде података, од прикупљања и аутоматске транскрипције подкаста до складиштења, индексирања и претраживања добијеног текста. Посебна пажња је посвећена специфичностима српског језика, укључујући паралелну употребу ћирилице и латинице, морфолошку сложеност и ограничену доступност језичких ресурса у односу на језике са већим бројем дигиталних ресурса.

Лексичка претрага је реализована помоћу *Elasticsearch*-а и алгорита *BM25*, уз прилагођени анализатор за српски језик. Овај приступ омогућава ефикасно проналажење садржаја на основу подударарања термина, али његова ограничења постају изражена код описних упита и случајева у којима корисник и релевантан транскрипт различитим ријечима изражавају исту информациону потребу.

Због тога је систем проширен семантичком претрагом, заснованом на *embedding* векторима модела *multilingual-e5-base*. Транскрипти су груписани у текстуалне *chunk*-ове, чије се векторске репрезентације чувају у *PostgreSQL* бази помоћу проширења *pgvector*. За ефикасно проналажење сличних вектора коришћени су *HNSW* индекс и косинусна удаљеност, која је директно повезана са косинусном сличношћу. На тај начин је омогућено проналажење тематски повезаног садржаја и када између упита и транскрипта не постоји непосредно лексичко подударарење.

Како лексичка и семантичка претрага имају различите предности и ограничења, њихове ранг-листе су обједињене примјеном *Reciprocal Rank Fusion (RRF)* методе. Тако је формирана хибридна претрага која користи информације добијене из оба приступа, без потребе за директним поређењем њихових различито дефинисаних скорова релевантности.

Сprovedена евалуација над 20 ручно означених упита је показала разлике између три приступа. Лексичка претрага је остварила просјечни ***Recall@10*** од 0,3257, ***MRR@10*** од 0,6667 и ***nDCG@10*** од 0,3889. Семантичка претрага је остварила веће вриједности све три мјере у односу на лексичку претрагу, са ***Recall@10*** од 0,5698, ***MRR@10*** од 0,9500 и ***nDCG@10*** од 0,6318. Хибридна претрага је остварила ***Recall@10*** од 0,5682, ***MRR@10*** од 1,0000 и ***nDCG@10*** од 0,6453. Семантичка претрага је, према томе, имала незнатно већи ***Recall@10***, док је хибридна претрага остварила највеће вриједности ***MRR@10*** и ***nDCG@10***. Посебно је значајан резултат ***MRR@10*** = **1,0000**, који показује да се за сваки од 20 тестираних упита бар једна релевантна епизода налазила на првој позицији хибридне ранг-листе.

Резултати показују да семантичка компонента значајно допуњује лексичку претрагу, нарочито код сложенијих упита формулисаних природним језиком. Истовремено, резултати хибридне претраге показују да комбиновање лексичких и семантичких сигнала може побољшати рангирање релевантних епизода, иако у спроведеној евалуацији није повећало просјечни ***Recall@10*** у односу на самосталну семантичку претрагу. Тиме је остварен систем који се не ослања искључиво на подударарење ријечи, већ користи и семантичку сличност садржаја.

Постоји више праваца у којима се реализовано рјешење може даље унаприједити. Проширење евалуационог скупа на већи број упита, канала и независних оцјењивача омогућило би поузданију процјену квалитета претраге. Додатно би се могло испитати подешавање параметра *RRF* методе, увођење минималног прага

семантичке сличности, примјена других начина фузије резултата и *cross-encoder reranking*-а. Простор за унапређење постоји и у начину формирања *chunk*-ова, означавању најрелевантнијих дијелова семантичких резултата и оптимизацији система за већи број истовремених корисника.

Развијени систем показује да се комбиновањем класичних метода информационог претраживања и савремених векторских репрезентација може изградити практичан претраживач за велики корпус говорног садржаја на српском језику. Очувањем временских информација транскрипта кориснику се омогућава да, поред проналажења релевантне епизоде, непосредно приступи дијелу разговора у којем се тражена тема појављује. На тај начин садржај који би иначе остао скривен у сатима аудио-видео материјала постаје доступан за претраживање, откривање и даљу употребу. На тај начин претрага не служи само проналажењу ријечи, већ и проналажењу идеја које се иза њих налазе.

Списак слика

Слика 1	Преглед припреме података и фаза индексирања	7
Слика 2	Преглед обраде корисничког упита и формирања хибридних резултата	8
Слика 3	Дијаграм класа модела података: <i>Channel</i> , <i>Video</i> , <i>TranscriptSegment</i> и <i>Chunk</i>	16
Слика 4	Алгоритам увоза података из <i>.jsonl.gz</i> датотека у <i>PostgreSQL</i> базу.	17
Слика 5	Поједностављен приказ процеса анализе текста у <i>Elasticsearch</i> -у	23
Слика 6	Алгоритам индексирања у <i>Elasticsearch</i> -у од провјере постојања индекса до провјере учитаних докумената	27
Слика 7	Алгоритам корисничке претраге у <i>Elasticsearch</i> -у	29
Слика 8	Алгоритам <i>batch embed</i> -овања корпуса и уписа векторских репрезентација у базу	39
Слика 9	Алгоритам векторске претраге	42
Слика 10	Алгоритам фузије резултата примјеном <i>RRF</i> методе	50

Списак листинга

Листинг 1	Конфигурација <i>PostgreSQL</i> базе са <i>pgvector</i> екстензијом	9
Листинг 2	<i>Dockerfile</i> за изградњу прилагођене <i>Elasticsearch</i> слике са <i>analysis-icu</i> додатком	10
Листинг 3	Конфигурација <i>nginx</i> -а за просљеђивање захтјева <i>Angular SPA</i> апликацији	10
Листинг 4	Команде за покретање и заустављање <i>Docker Compose</i> сервиса	11
Листинг 5	Примјер структуре JSON записа са метаподацима и транскрипцијом епизоде	14
Листинг 6	Покретање скрипте за увоз података унутар <i>backend</i> контејнера	16
Листинг 7	Конфигурација мапирања <i>Elasticsearch</i> индекса	22
Листинг 8	Примјер зауставних ријечи из уграђене листе за српски језик	24
Листинг 9	Конфигурација анализатора за индексирање текста на српском језику	25
Листинг 10	Конфигурација анализатора за претрагу са подршком за синониме	26
Листинг 11	Списак синонима коришћених у <i>Elasticsearch</i> претрази	26
Листинг 12	Резултат <i>bulk</i> индексирања докумената	28
Листинг 13	Конфигурација <i>match</i> упита за претрагу текста у <i>Elasticsearch</i> -у	30
Листинг 14	Дохватање метаподатака из релационе базе	31
Листинг 15	Нормализација оцјене релевантности и форматирање временских ознака сегмената	31
Листинг 16	Креирање <i>HNSW</i> индекса над <i>embedding</i> векторима	38
Листинг 17	Креирање <i>pgvector</i> екстензије у <i>Alembic</i> миграцији	38
Листинг 18	Покретање <i>batch embed</i> -овања корпуса унутар <i>backend</i> контејнера	40
Листинг 19	Спречавање дуплираног уписа <i>chunk</i> -ова у базу	40
Листинг 20	Резултат <i>batch embed</i> -овања цјелокупног корпуса	41
Листинг 21	Израчунавање косинусне сличности на основу косинусне удаљености	43
Листинг 22	Прикупљање и кеширање ранжираних листа <i>BM25</i> и векторске претраге	51
Листинг 23	Имплементација <i>Reciprocal Rank Fusion (RRF)</i> алгоритма за спајање ранжираних листа	51
Листинг 24	Покретање лексичке и векторске претраге за исти кориснички упит	52
Листинг 25	Пагинација резултата након <i>RRF</i> фузије	52
Листинг 26	Провјера филтера над резултатима хибридне претраге	53
Листинг 27	Одређивање величине скупа кандидата за векторску претрагу	54
Листинг 28	Спајање и хронолошко сортирање поклапања лексичке и векторске претраге	54

Списак табела

Табела 1	Списак канала у корпусу и број транскрибованих епизода по каналу.	13
Табела 2	Просјечни резултати евалуације лексичке, семантичке и хибридне претраге	58

Списак коришћених скраћеница

Скраћеница	Опис
ANN	<i>Approximate Nearest Neighbor</i> (приближна претрага к-најближих суседа)
API	<i>Application Programming Interface</i> (интерфејс за комуникацију између софтверских компоненти)
ASGI	<i>Asynchronous Server Gateway Interface</i> (стандард за асинхроне Python веб сервере)
ASR	<i>Automatic Speech Recognition</i> (аутоматско препознавање говора)
BM25	<i>Best Matching 25 — Okapi BM25</i> (алгоритам рангирања резултата претраге)
DCG	<i>Discounted Cumulative Gain</i> (мјера која узима у обзир релевантност резултата и њихову позицију у ранг-листи)
HNSW	<i>Hierarchical Navigable Small World</i> (графовски алгоритам/индекс за приближну претрагу к-најближих суседа)
HTTP	<i>Hypertext Transfer Protocol</i> (протокол за размјену података на вебу)
ICU	<i>International Components for Unicode</i> (библиотека за нормализацију Unicode текста)
IDCG	<i>Ideal Discounted Cumulative Gain</i> (вриједност DCG-а за идеално поређану ранг-листу)
JSON	<i>JavaScript Object Notation</i> (текстуални формат за структурисану размјену података)
JSONL	<i>JSON Lines</i> (формат гдје сваки ред датотеке представља један самосталан JSON објекат)
MIRACL	<i>Multilingual Information Retrieval Across a Continuum of Languages</i> (референтни скуп за евалуацију вишејезичке претраге)
MRR	<i>Mean Reciprocal Rank</i> (мјера заснована на позицији првог релевантног резултата)
nDCG	<i>Normalized Discounted Cumulative Gain</i> (мјера квалитета рангирања резултата претраге)
NFC	<i>Normalization Form Canonical Composition</i> (Unicode облик нормализације којим се канонски еквивалентни записи свде на јединствен облик)
NLP	<i>Natural Language Processing</i> (обрада природног језика)
ORM	<i>Object-Relational Mapping</i> (мапирање објектно-оријентисаних података у релациону базу)
RAM	<i>Random Access Memory</i> (радна меморија рачунара)
RRF	<i>Reciprocal Rank Fusion</i> (метода спајања ранжираних листи резултата)
SPA	<i>Single Page Application</i> (апликација чије се рутирање обавља на страни клијента)
SQL	<i>Structured Query Language</i> (језик за рад са релационим базама података)
TF-IDF	<i>Term Frequency-Inverse Document Frequency</i> (мјера важности термина у документу у односу на цијелу колекцију)
TREC	<i>Text REtrieval Conference</i> (истраживачка конференција за проналажење текста)
URL	<i>Uniform Resource Locator</i> (интернет адреса)

Списак коришћених појмова

Појам	Објашњење
Анализатор (енг. <i>analyzer</i>)	Ланац трансформација кроз који пролази текст прије индексирања или претраживања, састављен од карактер филтера, токенизатора и филтера термина.
Векторско уграђивање (енг. <i>embedding</i>)	Нумеричка (векторска) репрезентација текста добијена моделом машинског учења, таква да семантички сличан садржај буде представљен блиским векторима у векторском простору.
Вокабуларни јаз (енг. <i>vocabulary problem</i>)	Појава да двоје различитих људи, независно једно од другог, за исти појам или радњу бирају исти термин у мање од 20% случајева, услед чега упит и садржај који описују исту тему често не дијеле ниједан заједнички термин.
Гранулација сегмената	Ниво детаљности на који је садржај подјелен (нпр. на један или више сегмената) ради навигације до тачног или приближног тренутка унутар епизоде.
Густа (<i>dense</i>) векторска репрезентација	Векторска репрезентација текста код које су координате углавном различите од нуле, а њихова димензија није директно повезана са величином рјечника.
Еуклидско растојање (енг. <i>Euclidean distance</i>)	Мјера удаљености два вектора у простору код које мања вриједност означава већу сличност; за разлику од косинусне сличности, зависи и од норми вектора, а не само од њиховог смјера.
Идемпотентна операција	Операција која, ако се позове више пута са истим улазним подацима, даје исти резултат као да је позвана једном.
Индекс (<i>Elasticsearch</i>)	Логичка цјелина у <i>Elasticsearch</i> -у састављена од једног или више фрагмената, унутар које се физички чувају и индексирају документи.
Инвертовани индекс	Структура података у којој се, за разлику од класичног приступа, за сваки термин чува информација о документима у којима се тај термин појављује.
Језик са ограниченим ресурсима	Језик за који постоји релативно мала количина дигитализованих текстуалних и говорних података, што отежава тренирање и прилагођавање модела обраде природног језика (на супрот доминантним језицима попут енглеског).
Кластер (енг. <i>Cluster</i>)	Група чворова који заједно управљају подацима и ресурсима система.
Копија (енг. <i>Replica</i>)	Копија примарног фрагмента која служи као додатни ниво заштите података при отказу хардвера, а може учествовати и у обради захтјева за читање.
Корпус	Велика, структурисана збирка аутентичних језичких података (писаних или говорних), намјенски одабрана и узоркована тако да буде репрезентативна за одређени језик, често аотирана језичким метаподацима.
Косинусна сличност (енг. <i>cosine similarity</i>)	Мјера сличности два вектора добијена нормализацијом унутрашњег производа дужинама оба вектора, тако да резултат зависи искључиво од угла између њих, а не од њихове дужине; узима вриједности у интервалу $[-1, 1]$.
Лексема	Основна самостална јединица лексичког система која обухвата све граматичке облике и различита значења која једна ријеч може остварити.
Лексичка претрага	Начин претраге заснован на подударану тачних ријечи или термина између упита и садржаја (нпр. кроз индексирање кључних ријечи).

Мапирање (енг. <i>mapping</i>)	Дефиниција структуре података у <i>Elasticsearch</i> -у која одређује тип и начин индексирања појединачних поља документа.
Морфологија	Део граматике који се бави врстама ријечи, њиховом структуром и промјеном њихових облика.
Нормализација (вектора)	Поступак свођења дужине вектора на јединичну вриједност, при чему смјер вектора остаје непромијењен; над нормализованим векторима унутрашњи производ постаје еквивалентан косинусној сличности.
Подкаст	Аудио (или аудио-видео) садржај који се дистрибуира у виду епизода, доступан за преузимање или стриминг путем интернета.
Пост-филтрирање	Приступ по којем се филтери примјењују тек над већ формираном, рангираном листом резултата, уклањајући оне који не задовољавају задате услове без промјене међусобног поретка преосталих резултата.
Референтни тест (енг. <i>benchmark</i>)	Стандардизован скуп задатака и података који омогућава упоредиво мјерење перформанси различитих система или модела.
Ријетка (<i>sparse</i>) векторска репрезентација	Векторска репрезентација текста код које већина координата има вриједност нула.
Семантичка претрага	Начин претраге који упоређује значење упита и садржаја, а не само подударане тачних термина.
Стематизација	Поступак свођења различитих облика ријечи на њихову заједничку основу (стем), уклањањем одређених наставака и дијелова ријечи; добијени стем не мора представљати стварну ријеч у језику.
Токен	Основна јединица на коју токенизатор раздваја текст прије обраде моделом; може представљати ријеч, дио ријечи или други текстуални елемент, у зависности од коришћеног токенизатора.
Токенизација	Поступак раздвајања текста на мање јединице — токене — које се затим користе у даљој обради текста.
Унакрсни језички пренос знања	Пренос научених представа или образаца из модела тренираног на једном (обично доминантном) језику на други, мање заступљен језик.
Унутрашњи производ (енг. <i>dot product</i>)	Мјера сличности два вектора која зависи и од угла између њих и од производа њихових дужина (норми), због чега вектори веће норме могу остварити већи унутрашњи производ без обзира на смјер.
Фрагмент (енг. <i>Shard</i>)	Физичка јединица у којој <i>Elasticsearch</i> складишти дио података једног индекса; свака представља засебну инстанцу претраживачког механизма.
Фузија на нивоу епизоде	Приступ спајању резултата лексичке и семантичке претраге према заједничком идентификатору епизоде (<i>video_id</i>), примијењен умјесто спајања на нивоу појединачног сегмента или <i>chunk</i> -а, због различите грануларности на којој та два система претраге раде.
Хибридна претрага	Приступ претраживању који комбинује лексичку и семантичку претрагу ради постизања прецизнијих резултата.
Чвор (енг. <i>Node</i>)	Покренута инстанца <i>Elasticsearch</i> -а.
<i>Bi-encoder</i>	Приступ код којег се упит и текст кодирају независно у засебне <i>embedding</i> векторе, након чега се њихова сличност израчунава поређењем добијених вектора.
<i>Bulk API</i>	<i>Elasticsearch API endpoint</i> који омогућава извршавање више операција индексирања, ажурирања или брисања докумената у оквиру једног <i>HTTP</i> захтјева.
<i>Chunk</i>	Текстуална јединица формирана груписањем једног или више узастопних сегмената транскрипта до задате максималне величине, коришћена као јединица за генерисање <i>embedding</i> вектора у семантичкој претрази.

<i>Cross-encoder</i>	Модел који упит и текст кандидата обрађује заједно, директно процјењујући релевантност тог текста за дати упит; за разлику од <i>bi-encoder</i> приступа, овакав начин рада омогућава прецизнију процјену релевантности, али је рачунски захтјевнији и типично се примјењује за додатно рангирање (<i>reranking</i>) мањег броја претходно издвојених кандидата.
<i>Overfetch</i> и дедупликација	Стратегија претраге код које се преко индекса прво дохвата шири кандидатски скуп резултата, а тек се над тим знатно мањим скупом врши груписање и дедупликација по епизоди, чиме се избјегава губитак предности индекса услед рада над цијелом табелом.
<i>Top-k</i> претрага	Проблем проналажења k вектора из колекције који су најсличнији датом упитном вектору.
<i>Zero-shot</i> (услови/евалуација)	Примјена модела на податке из домена за који модел није посебно обучаваан нити прилагођаван.

Биографија

Татјана Гавриловић је рођена 22.07.1999. године у Милићима, Република Српска, Босна и Херцеговина. Дјетињство је провела у Дервенти, гдје је 2014. године завршила основну школу “19. Април” као носилац Вукове дипломе. Исте године уписала је Гимназију општег смјера у Дервенти, коју је завршила 2018. године, поново као носилац Вукове дипломе и ученик генерације. Током школовања учествовала је на бројним општинским и регионалним такмичењима из математике, физике и историје.

У јулу 2018. године уписала је Факултет техничких наука у Новом Саду, смјер Софтверско инжењерство и информационе технологије, који је завршила 2022. године одбраном рада на тему “Имплементација Analyze me - Write me апликације употребом *Mendix* развојног окружења”.

Радно искуство стицала је кроз двије стручне праксе — у компанији Intermino у Новом Саду (јул – август 2022, Salesforce) и у компанији Vega IT (новембар 2022), на позицији софтверског инжењера у области веб програмирања. Од 2023. године запослена је у компанији Openfyle на позицији софтверског инжењера, гдје ради и данас. До 2025. године радила је и као сарадник у настави на Факултету техничких наука у Новом Саду, гдје је држала вјежбе из предмета Основе програмирања, Инжењерство серверског слоја и Пословне информатике — при чему је, слично као и на дипломском раду, у оквиру Пословне информатике коришћена *Mendix* развојна платформа.

Мастер студије, смјер Софтверско инжењерство и информационе технологије, уписала је у септембру 2023. године на Факултету техничких наука у Новом Саду. Говори енглески језик. Њена стручна интересовања обухватају системе за претрагу, модел-управљане (енг. *model-driven*) системе, системе засноване на знању и правилима (енг. *rule-based*), као и моделовање софтвера уопште. Интересовање за модел-управљене приступе дијелом произилази из искуства стеченог кроз рад са *Mendix* платформом, и на дипломском раду, у оквиру ког је развила апликацију намјењену писцима аматерима за подршку у писању књижевног романа, од почетне идеје до завршеног дјела.

Литература

- [1] A. Tamborrino, “Introducing Natural Language Search for Podcast Episodes.” Accessed: August 19, 2026. [Online]. Доступно на <https://engineering.atspotify.com/2022/03/introducing-natural-language-search-for-podcast-episodes>
- [2] J. Karlgren *et al.*, “TREC 2021 Podcasts Track Overview,” in *Proceedings of the Thirtieth Text REtrieval Conference (TREC 2021)*, National Institute of Standards, Technology (NIST), 2021. doi: [10.6028/NIST.SP.500-335.podcast-overview](https://doi.org/10.6028/NIST.SP.500-335.podcast-overview).
- [3] “TREC Podcasts Track.” Accessed: August 19, 2026. [Online]. Доступно на <https://trecpodcasts.github.io/>
- [4] J. Guljaš and U. Jelenković, “Семантичка претрага судске праксе.” Accessed: August 19, 2026. [Online]. Доступно на <https://sudskapraksa.rs/>
- [5] ICEF and R. Centar, “COMtext.SR.” Accessed: August 19, 2026. [Online]. Доступно на <https://icef-nlp.github.io/COMtext.SR/>
- [6] S. Antonijević Ubois, “From Data Scarcity to Data Care: Reimagining Language Technologies for Serbian and other Low-Resource Languages,” *arXiv preprint arXiv:2512.10630*, 2025, doi: [10.48550/arXiv.2512.10630](https://doi.org/10.48550/arXiv.2512.10630).
- [7] V. Batanović, “Методологија рјешавања семантичких проблема у обради кратких текстова написаних на природним језицима са ограниченим ресурсима,” Doctoral dissertation, 2020.
- [8] M. Benjamin, “Hard Numbers: Language Exclusion in Computational Linguistics and Natural Language Processing,” *Proceedings of the LREC 2018 Workshop on Sustainability, Flexibility, Concreteness, and Comprehensiveness in Multilingual Lexicography*, 2018.
- [9] L. Tang and T. Vuković, “NLP for preserving Torlak, a vulnerable low-resource Slavic language,” in *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, 2025.
- [10] Одсек за стандардни језик, “Морфологија.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.lingvistickitermini.rs/pojmovnik/morfologija/>
- [11] I. Klajn, *Gramatika srpskog jezika*. Beograd: Zavod za udžbenike i nastavna sredstva, 2005, p. 263.
- [12] Одсек за стандардни језик, “Лексема.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.lingvistickitermini.rs/pojmovnik/leksema/>
- [13] Docker, “Defining and running multi-container applications with Docker Compose.” Accessed: August 23, 2026. [Online]. Доступно на <https://docs.docker.com/compose/>
- [14] pgvector, “pgvector: Open-source vector similarity search for Postgres.” Accessed: August 23, 2026. [Online]. Доступно на <https://github.com/pgvector/pgvector>
- [15] Elastic, “ICU analysis plugin.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/elasticsearch/plugins/analysis-icu>
- [16] Elastic, “Download X-Pack: Extend Elasticsearch and Kibana.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.elastic.co/downloads/x-pack>
- [17] FastAPI, “FastAPI documentation.” Accessed: August 23, 2026. [Online]. Доступно на <https://fastapi.tiangolo.com/>

-
- [18] Angular, “What is Angular?.” Accessed: August 23, 2026. [Online]. Доступно на <https://angular.dev/overview>
- [19] Docker, “Multi-stage builds.” Accessed: August 23, 2026. [Online]. Доступно на <https://docs.docker.com/build/building/multi-stage/>
- [20] J. R. Batista, *Learn OpenAI Whisper: Transform your understanding of GenAI through robust and accurate speech processing solutions*. Packt Publishing, 2024, p. 372.
- [21] A. Jain, “TF-IDF in NLP (Term Frequency Inverse Document Frequency).” Accessed: August 21, 2026. [Online]. Доступно на <https://medium.com/@abhishekjainindore24/tf-idf-in-nlp-term-frequency-inverse-document-frequency-e05b65932f1d>
- [22] S. Connelly, “Practical BM25 — Part 2: The BM25 Algorithm and its Variables.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/blog/practical-bm25-part-2-the-bm25-algorithm-and-its-variables>
- [23] C. Gormley and Z. Tong, *Elasticsearch: The Definitive Guide: A Distributed Real-Time Search and Analytics Engine*. O'Reilly Media, 2015.
- [24] Elastic, “How Full-Text Search Works.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/solutions/search/full-text/how-full-text-works>
- [25] Elastic, “Similarity module.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/guide/en/elasticsearch/reference/current/index-modules-similarity.html>
- [26] Elastic, “Language analyzers.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/text-analysis/analysis-lang-analyzer#serbian-analyzer>
- [27] Elastic, “ICU analyzer.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/elasticsearch/plugins/analysis-icu-analyzer>
- [28] Elastic, “Stop token filter.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/text-analysis/analysis-stop-tokenfilter>
- [29] Apache Lucene, “Serbian stop words (stopwords.txt).” Accessed: August 21, 2026. [Online]. Доступно на <https://github.com/apache/lucene/blob/main/lucene/analysis/common/src/resources/org/apache/lucene/analysis/sr/stopwords.txt>
- [30] Elastic, “Stemmer token filter.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/text-analysis/analysis-stemmer-tokenfilter>
- [31] Elastic, “Bulk index or delete documents.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.elastic.co/guide/en/elasticsearch/reference/current/docs-bulk.html>
- [32] Elastic, “Tune for indexing speed.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.elastic.co/docs/deploy-manage/production-guidance/optimize-performance/indexing-speed>
- [33] Elastic, “Match query.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.elastic.co/guide/en/elasticsearch/reference/8.19/query-dsl-match-query.html>
- [34] N. Borwankar, *Vector Databases: A Practical Introduction*. O'Reilly Media, 2026.
- [35] G. W. Furnas, T. K. Landauer, L. M. Gomez, and S. T. Dumais, “The Vocabulary Problem in Human-System Communication,” *Communications of the ACM*, vol. 30, no. 11, pp. 964–971, 1987, doi: [10.1145/32206.32212](https://doi.org/10.1145/32206.32212).
- [36] S. Bruch, “Foundations of Vector Retrieval,” *arXiv preprint arXiv:2401.09350*, 2024, doi: [10.48550/arXiv.2401.09350](https://doi.org/10.48550/arXiv.2401.09350).

- [37] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020, doi: [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [38] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual E5 Text Embeddings: A Technical Report,” *arXiv preprint arXiv:2402.05672*, 2024.
- [39] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [40] intfloat, “multilingual-e5-base.” Accessed: August 28, 2026. [Online]. Доступно на <https://huggingface.co/intfloat/multilingual-e5-base>
- [41] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins, “Sparse, Dense, and Attentional Representations for Text Retrieval,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 329–345, 2021, doi: [10.1162/tac1_a_00369](https://doi.org/10.1162/tac1_a_00369).
- [42] Elastic, “What is hybrid search? How it works and when to use it.” Accessed: August 29, 2026. [Online]. Доступно на <https://www.elastic.co/what-is/hybrid-search>
- [43] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Boston, MA, USA: ACM, 2009, pp. 758–759. doi: [10.1145/1571941.1572114](https://doi.org/10.1145/1571941.1572114).
- [44] OpenSearch, “Building effective hybrid search in OpenSearch: Techniques and best practices.” Accessed: August 29, 2026. [Online]. Доступно на <https://opensearch.org/blog/building-effective-hybrid-search-in-opensearch-techniques-and-best-practices/>
- [45] G. M. Rosa *et al.*, “In Defense of Cross-Encoders for Zero-Shot Retrieval,” *arXiv preprint arXiv:2212.06121*, 2022.
- [46] N. Thakur, N. Reimers, A. Rüchlé, A. Srivastava, and I. Gurevych, “BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021.
- [47] S. Bruch, S. Gai, and A. Ingber, “An Analysis of Fusion Functions for Hybrid Retrieval,” *ACM Transactions on Information Systems*, 2023, doi: [10.1145/3596512](https://doi.org/10.1145/3596512).
- [48] W. X. Zhao, J. Liu, R. Ren, and J.-R. Wen, “Dense Text Retrieval Based on Pretrained Language Models: A Survey,” *ACM Transactions on Information Systems*, vol. 42, no. 4, pp. 1–60, 2024, doi: [10.1145/3637870](https://doi.org/10.1145/3637870).
- [49] E. M. Voorhees, “The TREC-8 Question Answering Track Report,” in *Proceedings of the Eighth Text REtrieval Conference (TREC-8)*, Gaithersburg, MD, USA: National Institute of Standards, Technology (NIST), 1999, pp. 77–82.
- [50] K. Järvelin and J. Kekäläinen, “Cumulated Gain-Based Evaluation of IR Techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002, doi: [10.1145/582415.582418](https://doi.org/10.1145/582415.582418).